UNIVERSIDAD DE GRANADA

TRABAJO FIN DE GRADO

GRADO EN INGENIERÍA INFORMÁTICA

µPS Practice System

Desarrollo de un sistema operativo básico como recurso didáctico

Autor
Alejandro Machuca Jiménez

Directores
Pablo García Sánchez (tutor1)
Alberto Guillén Perales (tutor2)

Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación

—

Granada, 22 de abril de 2026

µPS Practice System: Desarrollo de un sistema operativo básico como recurso didáctico

Alejandro Machuca Jiménez

Palabras clave: Sistema Operativo, Sistema Didáctico, Rust

Resumen El objetivo del proyecto es el desarrollo de un sistema operativo simple que sirva de ayuda para comprender las bases del diseño y funcionamiento de los Sistemas Operativos.

Para ello se partirá de un programa capaz de ejecutar código binario que se usará para posteriormente definir un kernel básico, capaz de hacer las operaciones esenciales sobre el hardware, que a su vez se usará para poder gestionar la memoria del dispositivo así como los procesos que en él se ejecutarán.

Concretamente se partirá de un sector de arranque diseñado en assembly y acto seguido se creará un kernel básico con las syscalls necesarias para implementar la base de un sistema operativo funcional en un lenguaje de alto nivel.

El trabajo estará fundamentado en la investigación de la evolución de los sistemas operativos desde que surgieron hasta que alcanzaron características similares al objetivo final de este proyecto.

µPS Practice System:
Desarrollo de un sistema operativo básico como recurso didáctico

Alejandro Machuca Jiménez

Yo, **Alejandro Machuca Jiménez**, alumno de la titulación GRADO EN INGENIERÍA INFORMÁTICA de la **Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación de la Universidad de Granada**, con DNI 75941079R, autorizo la ubicación de la siguiente copia de mi Trabajo Fin de Grado en la biblioteca del centro para que pueda ser consultada por las personas que lo deseen.

Fdo: Alejandro Machuca Jiménez

Granada, a 22 de abril de 2026

D. **Pablo García Sánchez (tutor1)**, Profesor del Departamento Ingeniería de Computadores, Automática y Robótica de la Universidad de Granada.

D. **Alberto Guillén Perales (tutor2)**, Profesor del Departamento Ingeniería de Computadores, Automática y Robótica de la Universidad de Granada.

Informan:

Que el presente trabajo, titulado ***µPS Practice System, Desarrollo de un sistema operativo básico como recurso didáctico***, ha sido realizado bajo su supervisión por **Alejandro Machuca Jiménez**, y autorizamos la defensa de dicho trabajo ante el tribunal que corresponda.

Y para que conste, expiden y firman el presente informe en Granada a 22 de abril de 2026.

Los directores:

Pablo García Sánchez (tutor1) **Alberto Guillén Perales (tutor2)**

Agradecimientos

A mi madre y a mi padre, gracias por haber estado siempre ahí para mí cuando he necesitado cualquier cosa.

A mi hermana, gracias por aguantarme todos estos años.

A David Jándula Sánchez, gracias por aceptar debatir conmigo siempre que me ha hecho falta un punto de vista diferente.

A mis tutores del TFG, Pablo García Sánchez y Alberto Guillén Perales, gracias por aceptar un proyecto tan ambicioso y acompañarme a lo largo del camino.

A mi profesora de lengua de la ESO, Ana Hernández Hernández, gracias por enseñarme a pensar de manera crítica y a no tener miedo de aprender cosas nuevas.

A mi tutora de primaria Ana Galindo López, gracias por introducirme a las nuevas tecnologías y al desarrollo de proyectos.

A mis profesores del instituto Juan Manuel Ortiz Campillos y Octavio Sergio Gámez Lara, gracias por ver el potencial que había en mí y animarme siempre a desarrollarlo.

To the group Datateknologerna from Finland thank you for introducing me to a new culture and different ways of thinking and working.

A special thanks to Hugo Edvard Söderholm, Carl Nordenbrink, Odin Matias Röblom, and Joakim Stewen for welcoming me as one of your own.

To Mirek Kudinoff, thank you for always asking the right questions and reminding me that there's always more to learn.

A Louise Pavillon, gracias por hacer mi estancia en Finlandia mucho más amena.

A todos los que, de una forma u otra, han contribuido a este camino, gracias.

Índice general

1. Introducción **15**
1.1. Motivaciones 15
1.2. Conceptos Importantes 16
1.3. Objetivos específicos 16
OE01. Simular los componentes de un kernel 16
OE02. Proveer una interfaz simple 16
OE03. Crear casos de uso para poder probar los componentes 16

2. Estado del arte **17**
2.1. Sistemas Operativos Educativos 17
2.1.1. Minix 17
2.1.2. Xv6 18
2.2. Ideas clave 19

3. Análisis del problema **21**
3.1. Requisitos del Sistema Operativo 21
RQ01. Estructura sobre memoria principal 21
RQ02. Control sobre la memoria Secundaria (Sistema de Archivos) 21
RQ03. Control de interrupciones básicas 21
RQ04. Retroalimentación vía monitor 22
RQ05. Terminal básica para la interacción con el sistema . . 22
3.2. Punto de partida 22
3.3. Video 23
3.4. Interrupciones 23
3.4.1. Interrupciones de reloj 24
3.4.2. Interrupciones de teclado 24
3.4.3. Excepción doble 25
3.5. Gestión de memoria principal 25
3.5.1. Segmentación 26
3.5.2. Paginación 27
3.6. Gestión de memoria secundaria 29

3.6.1. FAT32 30
3.7. Terminal 31
3.8. Monitorización de componentes 31

4. Planificación, Metodología y Presupuesto 33
4.1. Introducción 33
4.1.1. Objetivo 33
4.1.2. Herramientas 33
4.2. Metodología 34
4.3. Análisis del tiempo 34
4.3.1. Total 34
4.3.2. Evolución 34
4.3.3. Despliegue del tiempo 35
4.4. Gastos 35
4.4.1. Costes de personal 35
4.4.2. Costes de material 35
4.4.3. Costes indirectos 36
4.4.4. Coste total 36

5. Diseño 37
5.1. Lenguaje 37
5.2. Punto de partida 38
5.2.1. Inicio 38
5.2.2. Final 39
5.3. Interrupciones 39
5.4. Controlador de vídeo 40
5.5. Memoria principal 41
5.6. Sistema de Archivos 43
5.7. Terminal 44
5.8. Depuración 45

6. Implementación 47
6.1. Ejecución 47
6.2. Estructura 48
6.2.1. Cargo.toml 48
6.2.2. x86_65-ups.json 49
6.2.3. tests 50
6.3. Código fuente 50
6.3.1. main 50
6.3.2. lib 54
6.3.3. vga_buffer 57
6.3.4. interrupts 63
6.3.5. memory 67
6.3.6. allocator 69

6.3.7. shell 70
6.3.8. serial 72
6.3.9. debug 74
6.3.10. Ejemplo 77
6.3.11. Sistema de Archivos 77

7. Pruebas **93**
7.1. Tests de integración 93
7.1.1. heap_allocation 93
7.1.2. stack_overflow 94
7.2. Tests unitarios 95
7.2.1. vga_buffer 95

8. Conclusiones y trabajos a futuro **97**
8.1. Objetivos alcanzados 97
8.2. Trabajos a futuro 97
8.3. Conclusión 98

Bibliografía **100**

Capítulo 1

Introducción

En este capítulo se buscará explicar de manera detallada y concisa, de dónde surge la idea, por qué el proyecto se decide hacer de esta manera y qué conceptos han sido los más importantes a la hora de llevar la idea a cabo.

1.1. Motivaciones

El proyecto como tal surge de una idea anterior, la idea original para este TFG consistía en implementar un sistema operativo desde dentro de si mismo, por la complejidad y la oportunidad de aprendizaje que esto prestaría, pero con el tiempo quedó en evidencia la falta de recursos prácticos en este tipo de proyectos.

Podríamos deducir fácilmente que los contenidos disponibles de manera *online* sobre diseño e implementación de sistemas operativos o *kernels* (de aquí en adelante se usará la palabra sistemas operativos para englobar a ambos, para evitar hacer la distinción cada vez que se mencionen) al ser programas muy extensos y cuyo ámbito está principalmente dominado por desarrolladores senior que se formaron cuando los propios sistemas estaban naciendo.

Las principales herramientas que podemos encontrar son, escasos tutoriales, documentaciones muy extensas con un lenguaje muy técnico y prácticamente sin ejemplos teóricos, y los manuales de los propios interesados en el desarrollo de estos sistemas como pueden ser Intel o ARM, pero sorprendentemente no es fácil encontrar una herramienta que sea capaz de simular de manera realista el funcionamiento y la interacción de los diversos componentes que conforman un sistema operativo.

Ante esta falta de recursos surge la idea de implementar esta misma herramienta, para proveer a los futuros desarrolladores que pudieran verse

intimidados por el carácter puramente teórico de los recursos que se pueden encontrar por internet, y que estos desarrolladores puedan encontrar en esta herramienta una manera práctica de poner a prueba sus conocimientos, o, en su defecto, adquirir estos de una manera interactiva.

1.2. Conceptos Importantes

A lo largo de este proyecto y de la implementación de los componentes que se han llegado a tratar, los conceptos más complejos que se han encontrado han sido:

- Paginación y sus implementaciones.
- Sistemas de archivos.
- Heap.
- Shell.

1.3. Objetivos específicos

OE01. Simular los componentes de un kernel

El sistema operativo debe simular los diferentes componentes que componen un sistema operativo, para ser capaces de detenerlos y obtener información sobre ellos, de esa manera los usuarios serán capaces de usar la herramienta de manera interactiva y obtener *feedback* sobre sus operaciones y componentes.

OE02. Proveer una interfaz simple

Debe implementarse una interfaz simple y sencilla, que permita recibir la información que estos componentes transmitan cuando se les solicite, y amigable con los usuarios, para que sea lo menos intimidante posible.

OE03. Crear casos de uso para poder probar los componentes

Se deben proveer también una serie de procesos que pongan en funcionamiento estos componentes específicamente o en conjunto, para poder observar el funcionamiento de todos ellos en situaciones que simulen casos reales y ejemplo significativos, con el fin de obtener la mayor cantidad de información cuando se usen.

Capítulo 2

Estado del arte

Este capítulo servirá a modo de revisión de soluciones existentes relacionadas con el aprendizaje de sistemas operativos didácticos. Analizar cómo otros proyectos han abordado problemas similares permite encontrar soluciones diferentes y opciones de mejora respecto a los mismo.

2.1. Sistemas Operativos Educativos

Dado que este proyecto es un sistema operativo educativo, una buena manera de aprender es viendo qué funcionalidades sistemas operativos educativos anteriores implementaron.

En cada caso, analizaremos cómo las diseñaron y extraeremos lecciones que nos permitan implementar un mejor sistema operativo.

2.1.1. Minix

Minix es un sistema operativo *microkernel* estilo Unix lanzado por primera vez en 1987 por Andrew S. Tanenbaum. Se diseñó como un clon de Unix que pudiera ejecutarse en una computadora casera.

Inicialmente se diseñó para ser un sistema para estudiantes de universidad, para poder aprender cómo estos sistemas funcionaban, aunque eventualmente se volvió en un sistema operativo *microkernel* de uso común[1].

Microkernel

Un sistema operativo *microkernel* trata de ejecutar el mínimo de código en modo privilegiado, por lo que la mayoría del código que se ejecuta estará en modo usuario, esto implica que el sistema será muy seguro ya que únicamente se ejecuta de manera privilegiada lo necesario.[2]

Esto también permite que ante el fallo de uno de los componentes o todo el resto del sistema necesite reiniciarse, puede reiniciarse únicamente el componente fallido.

Limitaciones

El hecho de tener que cambiar constantemente entre contextos implica una reducción en el rendimiento, siendo inferior en eficiencia a los sistemas monolíticos tradicionales.

2.1.2. Xv6

Xv6 es una re-implementación educativa de Unix V6, creada por el MIT para un curso de sistemas operativos. El objetivo era ofrecer un kernel pequeño y fácil de entender que los estudiantes pudieran leer, modificar y depurar en su totalidad[3].

La arquitectura de este sistema era un kernel monolítico sencillo, en el que todos los subsistemas viven en el espacio de direcciones privilegiado, el código estaba claramente dividido en una serie de archivos o módulos cada uno con su funcionalidad. Este sistema también implementa a remarcar: un planificador, memoria virtual con páginas de 4096B, un sistema de archivos tipo Unix V6, concurrencia y sincronización y *syscalls* básicas[3].

Este sistema operativo puede ser ejecutado en qemu lo que significa que es posible depurarlo usando el GDB, también implementa una serie de scripts de pruebas para comprobar que los sistemas funcionan correctamente y en general proporciona un balance entre una ejecución real y simplicidad, lo que lo convierte en una herramienta muy útil para los estudiantes que desean aprender cómo un sistema real funciona[3].

Kernel monolítico

Un sistema monolítico es un tipo de núcleo donde toda la funcionalidad principal se ejecuta en modo privilegiado, esto implica que la comunicación entre sistemas será mucho más sencilla que en un sistema microkernel.[4]

Un ejemplo de un sistema operativo que usa un kernel monolítico es Linux, que lleva usando este tipo de kernel desde su primera implementación.

Limitaciones

Pese a ser mejor en rendimiento comparado con un sistema microkernel, en el caso de que un servicio o controlador fallase al estar en el mismo espacio que el *core* puede ocurrir un fallo que haga caer al sistema.

2.2. Ideas clave

De estos sistemas podemos obtener varias ideas que son interesantes para nuestro sistema:

- **Microkernel vs Monolítico**

  Pese al riesgo de provocar un fallo catastrófico ante el fallo de uno de los componentes del sistema, la arquitectura monolítica provee una manera más sencilla de entender los sistemas, ya que no es necesario preocuparse por los múltiples cambios de contexto entre proceso que ocurren en un microkernel.[5]

  La arquitectura monolítica agrupa todo el códigos crítico en un único espacio de memoria, lo que permite comprender de manera más sencilla la interacción entre los archivos.

- **Simplicidad**

  El éxito de Xv6 demuestra que es posible crear un sistema que sea capaz de equilibrar funcionalidad real con una implementación simple de sus componentes. Por lo tanto este sistema tomará el mismo enfoque: implementar un sistema monolítico minimalista para facilitar el aprendizaje, y a la vez se incorporarán mecanismos que permitan obtener un *feedback* interactivo y depurar cada componente.

- **Conjunto de programas de prueba**

  Al igual que en Xv6 se implementarán una serie de programas que hagan uso de los componentes para poder comprobar que funcionan con normalidad, aunque de manera diferente a como se implementó en este sistema, se proveerá de una manera integrada de obtener retroalimentación **durante** la ejecución del componente.

Capítulo 3

Análisis del problema

En este capítulo se expondrá la información recopilada sobre los diferentes componentes de un sistema operativo básico, y las diversas maneras de implementarlas y la comparación entre ellas.

3.1. Requisitos del Sistema Operativo

RQ01. Estructura sobre memoria principal

El sistema debe proveer una estructura para la memoria principal (Tales como la paginación, la segmentación o los accesos directos a memoria) que permita interactuar con la memoria de manera simple y controlada.

Los requisitos definidos a continuación están destinados a reflejar de manera simple pero funcional los componentes fundamentales de un sistema operativo moderno. Se han elegido estos requisitos por su relevancia conceptual, su aplicabilidad a casos reales.

RQ02. Control sobre la memoria Secundaria (Sistema de Archivos)

El sistema debe proveer también de las herramientas que permitan leer y almacenar bloques de datos en memoria no volátil, con algún tipo de estructura (Como por ejemplo FAT32 o EXT4) para poder probar cómo estos componentes funcionan y poder guardar de manera permanente valores y/o en dado caso, programas.

RQ03. Control de interrupciones básicas

El programa debe ser capaz de tratar las interrupciones hardware y excepciones de CPU básicas para que un sistema funcione, tales como:

- Interrupciones de teclado (Para poder interactuar con el sistema)
- Interrupciones de reloj (Por ejemplo para la posible implementación de un planificador)
- Excepción doble (como *fallback* de cualquier interrupción que pudiese suceder y cuya gestión no haya sido implementada)

RQ04. Retroalimentación vía monitor

El sistema operativo tiene que ser capaz de devolver información de alguna manera, para una interacción en tiempo real con el sistema por parte de los usuarios y una monitorización, además de poder comprobar los mensajes de los componentes explicando su propio funcionamiento.

RQ05. Terminal básica para la interacción con el sistema

El proyecto debe tener algún tipo de programa básico tipo shell que implemente la lógica necesaria y los comandos básicos para que cualquier usuario sea capaz de probar a conveniencia los diferentes componentes y funcionalidades de un sistema operativo, además de ser capaces de refrescar la pantalla o interactuar de maneras diferentes a las preestablecidas con el propio sistema.

3.2. Punto de partida

Para poner al lector en situación, en un programa ordinario existen diversas herramientas que se pueden usar para su diseño:

- La librería estándar, que almacena valores y provee ciertas estructuras y funciones básicas.
- Varias capas de abstracción que evitan la necesidad de trabajar directamente sobre direcciones de memoria.
- Librerías y lenguajes de programación que funcionan sobre el kernel y permiten beneficiarse de esta estructura preexistente para simplificar los programas y el proceso de creación de los mismos.

A la hora de desarrollar un *kernel*, cualquier cosa que dependa de la librería estándar está fuera de la ecuación, por lo que las opciones que existen son:

- beneficiarse de librerías escritas por la comunidad que no hagan uso de la librería estándar. (Lo cual no es especialmente común ya que prácticamente todos las librerías de este estilo están escritas por la necesidad de desarrolladores de sistemas operativos por tener estas herramientas).

- escribir nuestras propias librerías desde cero, ya sea implementando partes de la librería estándar para hacerlas funcionar sobre nuestro diseño, o haciéndolas de tal manera que no necesiten en absoluto acceder a ellas.

Estas comparaciones ponen en evidencia el hecho de que, pese a que un sistema operativo es, en esencia, un programa igual a cualquier otro, escrito en un lenguaje de programación igual que cualquier otro y con los mismos componentes que cualquier otro, entra en realidad en una clasificación completamente diferente a la de un programa al uso.

3.3. Video

La implementación de los controladores de vídeo son especialmente importantes en un proyecto como este, sin una retroalimentación a través de un monitor el usuario está básicamente ciego, incapaz de interactuar en tiempo real con el sistema.

Debido a lo crucial de este componente es en esencia lo primero que se implementa, especialmente porque no sólo el usuario necesita de este componente para ser capaz de comprobar lo que pasa, sin ello el desarrollador se ve forzado a comprobar el estado de su máquina esencialmente mediante los valores de los registros y las flags (Que como se podrá deducir, en las primeras fases de un proyecto como este son más bien escuetos).

Para la implementación de los controladores de vídeo hay dos maneras principales de lograrlo:

- Implementar un controlador de vídeo que trabaje sobre la memoria de la VGA (Básico, primitivo y muy limitado, esencialmente recomendado para las primeras fases del proyecto). [6]
- Implementar un controlador de vídeo que haga uso de los *framebuffers* (Mucho más complejo y avanzado, permite cargar y definir fuentes, diversas resoluciones de pantalla...). [7]

3.4. Interrupciones

La implementación de la lógica que siguen las interrupciones queda totalmente a decisión del desarrollador, aunque algunas estructuras básicas como el uso de la IDT están definidos por la arquitectura sobre la que se implementa.

Una interrupción es, en esencia una señal, y esta señal está asociada a una

función (llamada handler), cuya dirección (al menos en x86_64) se almacena en una estructura conocida como la IDT (Interrupt Descriptor Table) donde se relaciona con la señal que representa a esa interrupción.[8]

Las interrupciones pueden ser activadas y desactivadas, lo cual es especialmente importante en procesos delicados como la inicialización del sistema operativo ya que es la porción del código en la que se reserva la memoria, se crean las estructuras básicas para el correcto funcionamiento del sistema y mucho más.

Una interrupción inesperada en este punto del sistema sería catastrófico, ya que podría evitar la inicialización de estos componentes necesarios, por este motivo lo que se hace es pausar las interrupciones.

La gestión de interrupciones también debe ser atómicas (no debería recibirse una interrupción mientras se está tratando otra) la CPU (o en un sistema multi-núcleo, el núcleo que la recibe) debe pausar su ejecución normal y ejecutar por completo la función que la IDT ha devuelto, esto se hace guardando los valores asociados al proceso que se estaba ejecutando de manera temporal, ejecutando la función en cuestión y posteriormente devolviendo los valores al estado previo a la interrupción.

Dicho esto hay tres interrupciones que nos interesan especialmente, son las mencionadas en las especificaciones, en orden de importancia:

3.4.1. Interrupciones de reloj

Las interrupciones de reloj son la interrupción más importante de un sistema operativo, quitando de la cuestión el hecho de que si no se manejan se van a recibir una cantidad infinita de excepciones dobles, los componentes como el planificador (excepto los basados en interrupciones como los periféricos) usan las interrupciones de reloj para coordinarse y funcionar, por ejemplo el temporizador APIC se encarga de lanzar las interrupciones de reloj.[**?**]

Cada interrupción de reloj, un sistema operativo que implemente procesos y planificación ejecuta una serie de comprobaciones, que deciden cuestiones tales como qué proceso va a ser el siguiente en ganar acceso a la CPU, decidir si es necesario liberar memoria o intercambiar procesos según la presión de memoria actual...

3.4.2. Interrupciones de teclado

Las interrupciones de teclado son interrupciones que se reciben cuando se pulsa una tecla en un teclado PS/2. Para un simulador, con esto es suficiente, aunque hoy en día la mayoría de componentes se conectan vía USB, por lo tanto, hay que usar una interrupción diferente, manejar una serie de meta-

datos (Tipo de dispositivo, valores de la llamada...) algunos dispositivos incluso implementan *handshakes* para la coordinación con el sistema.[8]

Algunos portátiles modernos utilizan el *hub* USB (el componente que centraliza todos los puertos USB de un dispositivo y se encarga de hacer las interrupciones) como su método de entrada para el teclado integrado, aunque aún se pueden encontrar algunos no tan antiguos que usan la interrupción de teclado como punto de entrada del teclado integrado.

Excepciones de la CPU Las excepciones se implementan en x86_64 como un tipo de interrupción, se definen también en la IDT y tienen reservado un espacio de las direcciones de la misma, sin embargo hay dos puntos clave a tener en cuenta respecto a las excepciones:

- Las excepciones son un tipo de interrupción, que están relacionadas con la CPU y se lanzan en casos en los que hay una operación o una situación específica que requiere de un trato especial, como divisiones entre 0 o el acceso a una página inexistente entre otras.[9]
- Las excepciones son interrupciones **software**, lo que quiere decir que se lanzan desde partes del código[10]

3.4.3. Excepción doble

Las excepciones dobles son un tipo de excepción que se debe implementar a la mayor brevedad posible, esto es debido a que estas excepciones se lanzan cuando una interrupción llega mientras se estaba ejecutando el código de otra interrupción, esto puede suceder en dos casos de manera más común:

- Cuando no se ha implementado una interrupción pero si se ha activado su señal, pues al no haber una implementación tampoco se recibe nunca una señal de final de interrupción ni es una implementación atómica por lo que permite la entrada de una interrupción diferente.[11]
- Cuando hay un fallo en la implementación de la interrupción y no se garantiza la atomicidad.

3.5. Gestión de memoria principal

Memoria principal es el nombre que se le da al espacio de almacenamientos en el que se almacenan los programas de un sistema operativo (incluso su propio código) durante la ejecución del mismo.

A día de hoy el término memoria principal está esencialmente reservado para la RAM, aunque históricamente este término también ha englobado muchos otros tipos de almacenamiento, incluyendo espacios no volátiles.

Aunque técnicamente podríamos acceder a la memoria principal sin utilizar ningún tipo de estructura, y podríamos consultar, escribir y manipular directamente su contenido, esto presenta algunas desventajas importantes:

- Falta de protección entre procesos. (Propenso a corrupciones)
- Uso de memoria física. (El hecho de usar memoria física explícitamente hace más complejo reordenar la memoria, cada proceso debe conocer exactamente su posición en la RAM)

Debido a estas desventajas, no es común que un sistema operativo trabaje directamente sobre las direcciones físicas de la memoria, y es preferible la implementación de una estructura de control que permita la manipulación controlada de las direcciones; para ello, se introduce el concepto de direcciones virtuales.

Espacio de direcciones virtuales El concepto de direcciones virtuales se crea para poder alojar los programas donde se necesite, imaginemos por ejemplo que se ha creado un programa y es necesario ejecutar este programa dos veces, si se trabaja sobre el espacio de direcciones físicas esto es sencillamente imposible si se utiliza el mismo código, porque se debe especificar de manera explícita qué posiciones reales va a ocupar este programa.[12]

Para resolver esto una solución es que las direcciones de los programas sean **relativas** (también conocidas como direcciones virtuales o lógicas), de esta manera al alojar el programa podemos decidir dónde se va a alojar, y calcular las posiciones físicas a partir de las posiciones relativas en función del espacio que los programas hayan ocupado al alojarse en la memoria principal.[9]

Hay diversas estrategias que se pueden utilizar para interactuar con la memoria principal mediante direcciones virtuales, pero principalmente se tienen en cuenta dos métodos:

3.5.1. Segmentación

La segmentación consiste en dividir la memoria principal en segmentos, cada posición de la memoria con un tamaño diferente dependiendo del espacio que necesite, y de manera contigua, esencialmente se alojan los programas donde quepan, permitiendo usar direcciones virtuales. El principal inconveniente de la segmentación es la aparición de la Fragmentación externa[13].

La fragmentación externa es lo que ocurre cuando, tras alojar una serie de programas y estos liberarse, quedan posiciones de memoria vacías, ya sea por ser demasiado pequeñas o por otros motivos, son posiciones de memoria que no van a poder ser utilizadas y hay que hacer uso de la *Desfragmentación*, proceso que consiste en copiar los valores de las direcciones de la memoria

principal y realojarlos de manera que no haya fragmentación[13].

El problema de la desfragmentación es lo increíblemente costosa que es, y la cantidad de direcciones a las que accede, resultando en muchísimas lecturas y escrituras, lo cual es lento y puede resultar dañino para la memoria.

3.5.2. Paginación

La paginación se basa en dividir todo el espacio de la memoria en bloques de igual tamaño, llamados marcos de página, lo común es que estos marcos tengan un tamaño de 4KiB[14].

Esta solución introduce el concepto de fragmentación interna, que por contraparte con la segmentación consiste en que, al reservar una página, la granularidad (es decir, el tamaño de las páginas) nos impida llenar estas páginas por completo, resultando en una pérdida de memoria, la diferencia con respecto a la segmentación es que cuando liberemos una página, esta fragmentación desaparece automáticamente sin tener que hacer uso de ningún algoritmo costoso como puede ser la desfragmentación[14].

Estas páginas se estructuran en tablas, estas tablas contienen la traducción a marcos de páginas de cualquier dirección que se quiera consultar, junto con una serie de flags que indican cosas como si un marco acepta lectura, escritura o ambas, si está deshabilitado, si está presente, si es ejecutable...

Las direcciones de memoria virtuales en paginación tienen la siguiente estructura:

- En las primeras posiciones se encuentra el offset dentro de la página, en el caso de que sean páginas de 4KiB (como es lo común) este offset se compone de 12 bits.
- Posteriormente se encuentra la dirección de la página a la que pertenece, puede ser directamente una dirección o, en el caso de las tablas multi-nivel, una serie de bits reservados por cada nivel.

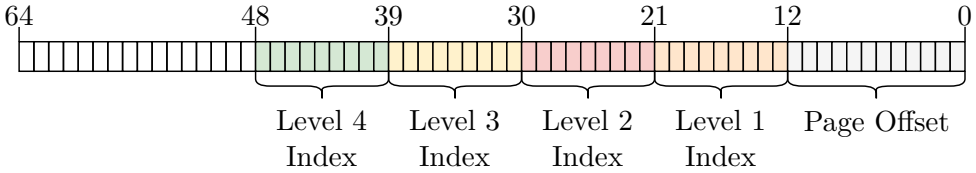

Figura 3.1: Representación de la estructura de una dirección de memoria usando paginación multinivel. Fuente: [15]

La paginación en sus inicios consistía de una única tabla, dentro de la cual tenemos todas las direcciones físicas de cada una de las entradas, pero existe un problema en esta solución, como indicamos antes los programas se es-

tructuran haciendo uso de las direcciones virtuales, y en cuanto a memoria física esto no es un problema, ya que cada vez que se quiere alojar un bloque de memoria, se busca un marco de página libre. Sin embargo la tabla tendrá tantas entradas como páginas haya entre los bloques de un programa.

El siguiente ejemplo se ha extraído del tutorial *online Writting an OS in Rust* [15]

Encontramos un programa que tiene 2 segmentos, uno con 50 bytes en la posición 0 y otro con 150 en la posición 100000 que corresponde a la página $\lfloor 100000/50 \rfloor = 2000$, imaginemos también que cada página puede almacenar únicamente 50 bytes, esto significaría que para un único programa hemos creado en la tabla de páginas 2003 entradas, la primera en el 0, a continuación 1999 entradas vacías y después 3 entradas más.

Como se puede ver se está almacenando una gran cantidad de memoria de manera ineficiente, dado que no podemos ignorar estas posiciones virtuales (porque de hacerlo la CPU ya no sería capaz de hacer la traducción de manera correcta), se deduce que tener una única página no es una opción viable para casos como este, para ello se crearon las **Tablas de Páginas multi-nivel**

Una página esencialmente no es más que información dentro de un marco, por lo que al hacer la traducción lo único que se hace es acceder a la posición en memoria de cada página en cada nivel, de esa manera se va traduciendo nivel a nivel y finalmente se obtienen los datos deseados después de unos cuantos ciclos.

El hecho de introducir varios niveles de páginas permite que tengamos datos muy alejados entre ellos con muchas menos entradas. En esencia cada nivel se encarga de un cierto número de marcos de página, por lo tanto si se usaran dos niveles de tablas, únicamente harían falta tres páginas para acceder a datos que estén en el dominio de dos páginas de nivel uno diferentes, la página de nivel 2 y dos páginas de nivel 1[14].

Esto ya hace que esta implementación sea mucho más eficiente en lo relativo a memoria que la implementación de una única tabla, sin embargo aún se pierde una gran cantidad de segmentos, por lo que en los sistemas modernos se suelen utilizar 4 niveles.

Es interesante remarcar que las arquitecturas x86_64 **necesitan** que la paginación esté activada para poder encenderse en modo extendido [1], por lo tanto para un sistema operativo que quiera utilizar palabras de 64 bits en esta arquitectura es obligatorio hacerlo mediante paginación.

[1] al menos a nivel básico, más adelante podría implementarse por ejemplo segmentación sobre esta paginación inicial, aunque probablemente no sería una buena idea

La paginación hace uso de un TLB (Translation Lookaside Buffer), que es una caché de hardware [2]que es mucho más rápida que la memoria principal, en la que se anotan la traducción de memoria virtual a memoria física de las páginas a las que se ha accedido recientemente[16].

La Unidad de Acceso a Memoria (MMU) pide la traducción al TLB, si hay un *hit* o acierto, la devuelve directamente, si hay un *miss* o fallo, procede a hacer la traducción de manera normal, esto le da a la paginación un aumento de velocidad considerable respecto a un sistema que no hiciera uso del buffer[16].

Esto es importante porque por cada nivel de tablas que se use, se añaden varios ciclos a la traducción [3], lo que la hace más lenta con respecto a las otras implementaciones, aunque en cuestión de memoria mucho más eficiente.

El uso del TLB es muy importante para permitir a la paginación competir en velocidad contra las demás soluciones, ya que el uso del TLB permite a la paginación resolver la traducción en un par de ciclos, en lugar de tener que hacer todas las traducciones de páginas cada vez.

3.6. Gestión de memoria secundaria

Se conoce como memoria secundaria la memoria no volátil que se utiliza para almacenar datos de manera permanente[4], ejemplos de esto son los HDD o los SSD

Para la organización de estos dispositivos existen una gran variedad de Sistemas de Archivos. Entre ellos podemos señalar FAT (cualquiera de ellos) como uno de los más importantes, ya que además de ser simple lleva en la industria prácticamente desde que se creó.

También existen otros como EXT, xfs, otros especializados como linux-swap, usado para las particiones SWAP usadas para alojar más memoria virtual que la memoria principal física existente en un dispositivo.

Los beneficios de utilizar un sistema de archivos sobre modificar directamente los sectores de un dispositivo de memoria secundaria (también llamada memoria masiva) son:

- **Uso de estructuras de alto nivel**

[2]Caché compuesta exclusivamente por transistores, que a diferencia de la RAM que hace uso de condensadores son mucho más rápidos

[3]Podría pensarse que es únicamente un ciclo, por el hecho de que cada nivel que se añade, aumenta el número de traducciones en uno, pero debemos tener en cuenta la latencia que añaden los accesos a memoria

[4]O semi-permanente, no se mantiene durante toda la eternidad pero si se mantiene durante la vida útil del dispositivo se considera permanente (Un hdd puede durar 2 años sin pérdidas de información significativas)[17]

El uso de un sistema de archivos permite utilizar estructuras de alto nivel como archivos y directorios, que hacen que los usuarios puedan almacenar datos o crear estructuras para sus sistemas sin tener que entender las implicaciones que esto tiene sobre la memoria secundaria.

- **Compatibilidad entre dispositivos**

  Al usar un sistema de archivos simplemente tenemos que definir cómo un dispositivo lee y escribe. Toda la interacción con los sectores que se extraen o se alojan en esta memoria se puede extrapolar e implementarse una única vez.

- **Protección de datos**

  Cuando se hacen los accesos a través de un sistema de archivos, las diferentes entradas del mismo están protegidas de accesos por parte de otras. Al alojar sus datos no se permite modificar estos datos si no se hace de manera directa, por lo que no existe la posibilidad de corromper accidentalmente la memoria de un archivo o directorio

3.6.1. FAT32

El sistema de archivos FAT32 consiste en crear una tabla, llamada FAT (File Allocation Table) que contendrá datos relacionados con cada archivo o directorio. El dispositivo que aloja el sistema de archivos se divide en sectores, la unidad de almacenamiento básica en FAT32. Un grupo de sectores (de número configurable) se conoce como clúster. [18]

Las entradas de una tabla apuntan a la siguiente entrada del archivo o directorio, en caso de que un archivo no quepa en un clúster, reservaríamos dos, y el último clúster de la cadena apunta a EOC (End Of Cluster)[18].

Un directorio es, en esencia, un archivo que contiene entradas de 32 bytes, cada una de ellas siendo la información del elemento que ocupa esa posición, una entrada (conocidas como DirEntries) contiene información como el tamaño del archivo, el primer clúster de la cadena, el nombre del archivo junto con su extensión...[18]

Un archivo es simplemente un conjunto de datos, que puede tener estructura (por ejemplo si se usa algún tipo de extensión que especifique un formato) o ser simplemente texto binario escrito sobre la posición del archivo.

En FAT32 se suelen alojar dos tablas, como medida de redundancia, para detectar errores o corrupciones, y un número de sectores reservados. Además, al inicio de las particiones se almacenan dos estructuras:

- **BPB (Boot Partition Block)[18]**

Contiene datos de inicialización como el número de sectores por clúster, el número de bytes por sector, el número de tablas, el número de sectores por tabla...

- **EBR (Extended Boot Record)[18]**

  Contiene datos de inicialización pero para FAT desde el 12 hasta el exFAT, en él se encuentra la versión del FAT, el clúster del root...

Los clúster en FAT32 se encuentran directamente después de la última tabla, dado que los dos primeros sectores de las tablas no son realmente entradas de archivos, el primer clúster pertenece a la tercera entrada de la tabla (con id 2 ya que empieza en 0). Cuando se quiera calcular el offset de un clúster se deben restar esas dos primeras entradas, debido a que la entrada 2 estará en la posición 0, la entrada 3 en la posición 1...[18]

Al borrar una entrada de un directorio no se pone a 0, dado que la primera entrada 0 se considera la última en un directorio, sino que se usa un valor especial para que se compruebe que la entrada se debe ignorar sin considerarla la última. Una entrada borrada se considera también libre, por tanto se puede alojar información en ella.

3.7. Terminal

Para la terminal en un sistema operativo que implemente procesos y planificación lo que se haría es un programa que pueda acceder a un directorio que contenga a su vez otros programas (en linux se utiliza el /bin).

Otra opción para un sistema operativo más básico sería implementar un emulador de terminal, que tenga funciones en lugar de programas y que acepte una serie de palabras reservadas, cada una de ellas ejecutando una función que permita acciones básicas como crear archivos y mostrar el sistema de archivos entre otras.

La creación de una terminal o un emulador de terminal es especialmente importante para este tipo de proyecto, ya que aunque sería posible hacerlo mediante argumentos y ejecutar estos programas modificando un archivo por ejemplo, es mucho más práctico tener la libertad de probar las cosas de manera manual, interactuando con el sistema de una manera más libre.

3.8. Monitorización de componentes

Por el mismo motivo que lo mencionado en el punto anterior, es especialmente importante para un sistema operativo didáctico como lo es este proyecto, ya que la idea detrás de él es poder pausar la ejecución de los componentes y

mostrar mensajes explicativos que permitan al usuario comprender qué está ocurriendo en cada uno de los componentes.

Esto se puede hacer por ejemplo, guardando los mensajes de log de los componentes antes de llamar a una función que imprimirá el mensaje y esperará hasta que el usuario dé la orden de continuar para seguir ejecutándose. Esto es especialmente sencillo en un sistema mono-programado donde únicamente hay espacio para una función en el único núcleo que hay en ejecución.

Capítulo 4

Planificación, Metodología y Presupuesto

Los costes temporales, económicos y las diferentes etapas de la implementación a lo largo del tiempo serán explicadas en profundidad da lo largo de este capítulo, para justificar la facturación estimada y comparar la estimación inicial del tiempo con los valores finales obtenidos.

4.1. Introducción

4.1.1. Objetivo

Para asegurar un avance constante y detectar qué partes del desarrollo han sido las más costosas, se ha decidido hacer uso de una herramienta de registro del tiempo desde el inicio del proyecto. El objetivo era contabilizar el esfuerzo que cada etapa del TFG ha requerido y compararlo con la planificación inicial.

4.1.2. Herramientas

Se ha utilizado la herramienta Clockify por la facilidad de uso y la capacidad que tiene para generar gráficos del tiempo registrado de maneras flexibles y estéticas.

Para manejar el desarrollo y los objetivos del proyecto se ha hecho uso de una lista de quehaceres (en inglés *TO-DO list*) en markdown, donde se han ido anotando los objetivos.

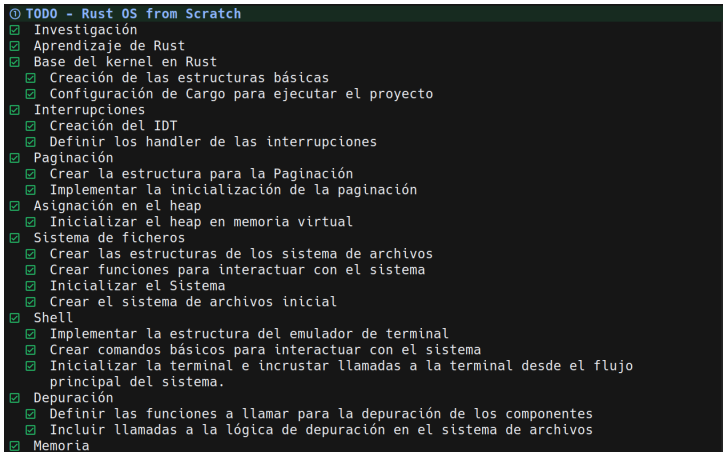

Figura 4.1: Captura de la lista de objetivos utilizada durante el proyecto

4.2. Metodología

Durante el desarrollo de este proyecto se ha seguido una metodología lineal, en la que se ha ido completando cada parte del proyecto antes de continuar con la siguiente (un ejemplo de ello es el desarrollo de los diversos componentes antes de desarrollar la Memoria del TFG).

Se ha seguido un desarrollo iterativo guiado por las necesidades del proyecto, comenzando por la implementación de los componentes básicos para el resto del sistema, y avanzando progresivamente hacia aquellos módulos dependientes o de nivel superior.

4.3. Análisis del tiempo

4.3.1. Total

En total durante el desarrollo del proyecto se han invertido 97 horas, 47 minutos y 13 segundos (97:47:13), cifra que demuestra el esfuerzo acumulado entre marzo y junio. Todo el tiempo registrado corresponde a este proyecto, sin distracciones en otros proyectos oficiales.

4.3.2. Evolución

Como podemos observar en el gráfico, conforme los meses se acercaban a la fecha de entrega la cantidad de horas invertidas en el proyecto se incrementaba, lo cual coincide con el final de las clases y un aumento en el tiempo disponible para dedicarlo en el proyecto.

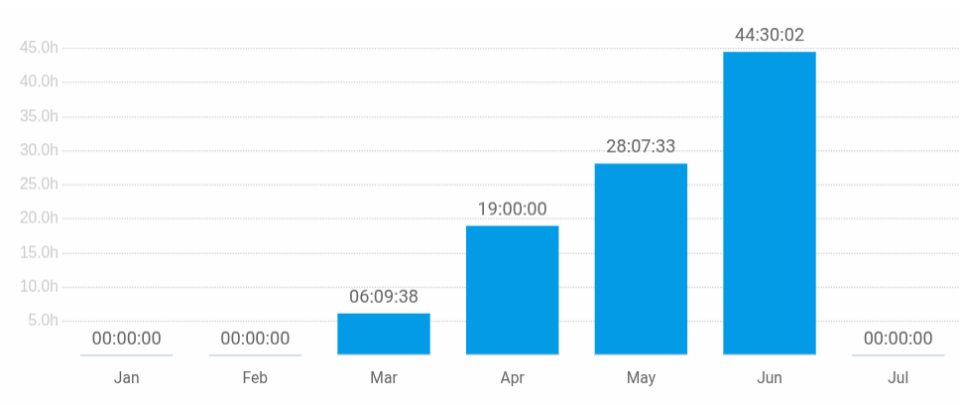

Figura 4.2: Evolución de la cantidad de horas invertidas en el proyecto a lo largo de los meses

4.3.3. Despliegue del tiempo

Como podemos observar en la lista anterior la cantidad de tiempo invertido en la implementación del código ha sido sustancialmente superior a la cantidad invertida en la memoria, sin embargo la memoria ha sido la actividad que más tiempo ha tomado por sí sola.

4.4. Gastos

4.4.1. Costes de personal

- **Desarrollador junior:** Tarifa: 25 €/h, 20 h/semana × 10 semanas = 200 h

$$200\,\text{h} \times 25\,\tfrac{€}{\text{h}} = 5\,000\,€$$

- **Tutores (2 personas):** Tarifa: 40 €/h, 2 h/semana × 6 semanas = 12 h cada uno

$$12\,\text{h} \times 40\,\tfrac{€}{\text{h}} = 480\,€ \text{ por tutor}$$

  Total tutores:

$$2 \times 480\,€ = 960\,€$$

4.4.2. Costes de material

- **Portátil ASUS ROG Strix G15:** Precio de compra: 1 600 €, vida útil: 4 años → amortización anual = 1 600 € / 4 años = 400 €/año

$$\text{Duración del proyecto} \approx 10 \text{ semanas} = 0{,}192\,\text{años}$$

$$400\,\tfrac{€}{\text{año}} \times 0{,}192\,\text{años} \approx 77\,€$$

Most tracked activities	All
Memoria • TFG	25:54:00
Disercion del TFG • TFG	13:45:27
Base kernel in Rust • TFG	13:36:58
Aprendizaje de Rust • TFG	10:00:00
File System • TFG	09:57:31
Paging • TFG	06:18:09
Starting research • TFG	06:09:38
Interruptions • TFG	05:50:00
shell • TFG	02:28:00
Heap alloc • TFG	02:22:26
Depuración • TFG	01:25:04

Figura 4.3: Cantidad de tiempo invertido en cada fase del proyecto

- **Herramientas de software libre y Linux:** 0 €

4.4.3. Costes indirectos

- **Alquiler (gastos incluidos):** 400 €/mes × 2,5 meses 1 000 €

4.4.4. Coste total

$$\begin{aligned}
\text{Total personal} &= 5\,000 + 960 = 5\,960\,€,\\
\text{Total material} &= 77\,€,\\
\text{Despliegue} &= 0\,€,\\
\text{Indirectos} &= 1\,000\,€.
\end{aligned}$$

$$\boxed{\text{Coste total} = 5\,960 + 77 + 0 + 1\,000 = 7\,037\,€}$$

Capítulo 5

Diseño

Para implementar este proyecto se han tomado diversas decisiones sobre la implementación de los diferentes componentes, el lenguaje que se usa para implementarlo, el punto de partida...

La idea de este capítulo es dejar claro para el lector por qué se han tomado las decisiones que se han tomado y qué ventajas tienen sobre el resto de opciones que se presentaban durante la implementación.

5.1. Lenguaje

Para la implementación del proyecto se ha decidido utilizar el lenguaje de programación Rust. Esta se debe principalmente a su popularidad en los últimos años, la simplicidad que brinda sobre C a la hora de aprenderlo (ya que el propio lenguaje ofrece un tutorial escrito por sus propios desarrolladores) y la rigurosidad del compilador que impone comprobaciones estrictas en tiempo de compilación que garantizan la seguridad de la memoria especialmente en código de bajo nivel.

Otro motivo de peso es que en los últimos años, partes del *kernel* de Linux han sido traducidas para utilizar Rust[19], por tanto el hecho de que el sistema operativo más utilizado (teniendo en cuenta teléfonos, servidores, embebidos...) considera que es un lenguaje beneficioso para implementar su *kernel* infunde confianza en él para la implementación de un *kernel* desde cero.

Este lenguaje de programación además ofrece un control muy directo sobre la memoria, permitiendo no solamente trabajar sobre posiciones virtuales directamente, sino que también permite decidir cómo se van a guardar las estructuras sobre la memoria (por ejemplo podemos usar repr(transparent)

para crear estructuras que únicamente contienen los datos, sin meta-datos, y alineados en memoria de manera exacta a como los introdujimos. Además podemos usar `repr(C)` para indicar a Rust que queremos que nuestra estructura tenga el mismo formato que lo tendría si se trabajara con el lenguaje de programación C.

Es precisamente este control sobre las estructuras, su formato y cómo se tratan los datos lo que viene especialmente bien a la hora de diseñar un kernel, donde las estructuras deben ocupar el mínimo posible y la memoria debe estar lo más compacta y organizada que se pueda.

Otro beneficio de Rust sobre C es que el compilador de Rust trata los errores comunes con menos permisividad que el compilador de C. Un ejemplo de esto es la memoria, en C se pueden dar fugas de memoria cuando hemos reservado memoria y no la hemos liberado al final[20], en Rust esto es mucho más complicado. El compilador se encarga de que cuando reservamos memoria, no se compile el programa si no se ha liberado o manejado de una manera correcta.

Además de todo lo mencionado anteriormente, hay que remarcar que Rust no tiene nada que envidiar a C en velocidad, ya que se ha demostrado un rendimiento similar en ambos lenguajes. Añadir que el compilador de Rust permite escribir código en C que luego será compilado, por lo que es posible beneficiarse de las situaciones en las que C funciona mejor que Rust, simplemente utilizando C en su lugar[21].

5.2. Punto de partida

Tras la elección de Rust como el lenguaje de este proyecto se decidió hasta dónde se iba a avanzar, puesto que un sistema operativo puede empezarse a desarrollar desde el *bootloader* y puede terminarse cuando el proyecto tiene soporte para varias arquitecturas, control para técnicas de almacenamiento avanzadas etc.

Por este motivo, debemos tener claro el inicio y el final:

5.2.1. Inicio

Este proyecto se construye sobre la mejor herramienta que se ha encontrado para el desarrollo de el mismo, un tutorial llamado **Writing an OS in Rust**[22] el cual comienza utilizando una *crate* (nombre que reciben las librerías en Rust) llamada *bootloader*. Esta librería implementa algunas necesidades básicas una paginación inicial, y el salto a modo protegido.

Aunque este tutorial nos servirá como base, se aplicarán variaciones que añadan funcionalidad sobre lo que provee, de esta manera esencialmente

convirtiendo el producto que nos enseña a construir, en nuestra propia creación.

5.2.2. Final

Para este Trabajo de Fin de Grado en específico se realizará la implementación de un sistema mono-programado que implemente una serie de comandos básicos, una terminal, paginación, un sistema de archivos y controladores de video.

Consideramos que lo realizado es suficiente para cumplir los objetivos y proveer de una versión inicial del sistema, sin embargo en el futuro se podrían implementar muchas otras características como se explicará en el capítulo *Conclusiones y trabajos a futuro*.

5.3. Interrupciones

En lo relacionado a las interrupciones se han implementado las cuatro interrupciones mencionadas en la sección de análisis y la interrupción de fallo de página, esta interrupción salta cuando intentamos acceder a un marco de página que no ha sido alojado.[20]

Las interrupciones como se explicó en el capítulo de análisis, se manejan creando una tabla conocida como IDT, en ella se relacionan las señales de interrupción con la dirección en la que se encuentra la función *handler*.

Otro dato importante es que para que un proceso se pueda ejecutar (Incluyendo el proceso principal en este sistema mono-núcleo) en x86_64 en modo protegido o extendido, es necesario definir una GDT (Global Descriptor Table), esto podría hacerse perfectamente definiendo una estructura GDT y plasmándola en memoria de manera estática.

- **Excepción de página**

  La excepción de página imprime por pantalla los datos relacionados con la página que ha fallado, después mantiene la CPU ociosa. Se ha implementado de esta manera porque según la implementación que se ha seguido no debería ocurrir esta excepción.

- **Excepción doble**

  La excepción doble se ha implementado de manera similar a las excepciones de página, de manera que simplemente devuelve un mensaje informando de lo relacionado con el causante de esta excepción, posteriormente no devuelve el control ya que, como hemos dicho, no debería ocurrir nunca esta excepción en la implementación actual.

- **Interrupción de teclado**

  La interrupción de teclado esencialmente lo que hace es traducir el carácter que se ha enviado, si es un carácter que lanzaría una nueva línea, se ejecuta el comando que se haya introducido al momento, dándole el control al emulador de terminal para que haga lo necesario para procesarlo, y posteriormente imprime lo devuelto.

  En caso de que fuese un borrado, se ordena al controlador de la pantalla borrar y se hace lo mismo al emulador de terminal, que lo borrará de su buffer interno.

  Cualquier otro carácter se envía al emulador de terminal y posteriormente se imprime en pantalla.

- **Interrupción de reloj**

  La interrupción de reloj informa de que se ha terminado sin hacer nada más con ello, de esta manera no interfiere con el funcionamiento normal de la CPU pero queda establecido si en algún momento se quisiera implementar la lógica relacionada a los procesos y el planificador.

5.4. Controlador de vídeo

Como se comentó en el capítulo de Análisis, respecto a los controladores de vídeo se puede crear un driver que trabaje directamente sobre las direcciones de memoria de la VGA, o crear un driver que trabaje utilizando *framebuffers*. Para este proyecto y por cuestiones de simplicidad se ha decidido utilizar un driver para la VGA, dado que es mucho más simple y sencillo de explicar.

Al tomar esta decisión se limita la resolución del monitor a `640×480` y los colores a 16 bits, sin embargo permite un control bastante amplio sobre el monitor y es suficiente para las tareas que se van a realizar en el sistema.

Para la implementación de este driver es necesario definir las estructuras que van a conformarlo y las funciones que van a permitir controlarlo, también definiremos una manera global de escribir para la depuración (en Rust específicamente encontramos esto en las macros `print!()` y `println!()`.

Este driver esencialmente escribirá caracteres en la memoria reservada para la VGA, que a su vez imprimirán estos caracteres automáticamente. Esto se logra definiendo métodos que accedan a esta memoria de manera segura y que, por ejemplo, al llegar al final de la línea, mueva el contenido de todas las líneas hacia arriba y haga hueco para una línea nueva.

En el capítulo de implementación se explicará con más profundidad cómo se ha logrado que este driver funcione y qué partes son importantes para

aprender cómo funciona.

5.5. Memoria principal

En lo relativo a la memoria principal se ha implementado paginación, dado que entre todos los mecanismos era el que más beneficios tenía, además de ser obligatorio para x86_64 en modo extendido.

La *crate* mencionada anteriormente, *bootloader* ya define la paginación por su cuenta, una simple pero suficiente para la inicialización de la arquitectura, sin embargo esto no es suficiente para el proyecto, ya que se busca implementar manualmente la paginación.

Debido a la implementación de la paginación de esta *crate* aparece un problema, las direcciones que se han estado usando en el proyecto antes de la implementación de este componente han sido direcciones virtuales, el único motivo por el que la VGA funciona es debido a que la *crate* enlaza la dirección virtual `0xb8000` en esta paginación que se menciona, a la dirección física `0xb8000`, por ello se escribe en la posición correcta.

Para solucionar este problema existen varias soluciones, pero nos centraremos en dos:

- **Acceso recursivo en la tabla de último nivel**

  Como se mencionó anteriormente, las páginas devuelven posiciones de memoria física sobre las que podemos leer, escribir y tratar como nos parezca, y siempre se hace una traducción por nivel, es precisamente este hecho lo que permite implementar paginación sobre la paginación inicial.

  Imaginemos que se busca acceder a la página de nivel 2 para añadir una entrada nueva, haría falta hacer únicamente 2 traducciones, la de nivel 4 y la de nivel 3, se obtendría la dirección física en la que se encuentra la pagina de nivel 2. En lugar de hacer únicamente 2 traducciones, lo que se hace es incluir en la página de nivel 4, un puntero a si misma.

  Si quisiéramos acceder al nivel 2 por ejemplo, lo único que habría que hacer es acceder dos veces al puntero de la página 4 a si misma, y la tercera y cuarta traducción nos llevarían, respectivamente, a la tabla de nivel 3 y de nivel 2, pudiendo así editar la información.

- **Mapeado directo de todas las páginas**

  Otra solución sería, por cada página que queramos acceder, en una posición muy lejana de la memoria principal, añadir un mapa que contenga en cada momento, a qué posición física apunta cada dirección

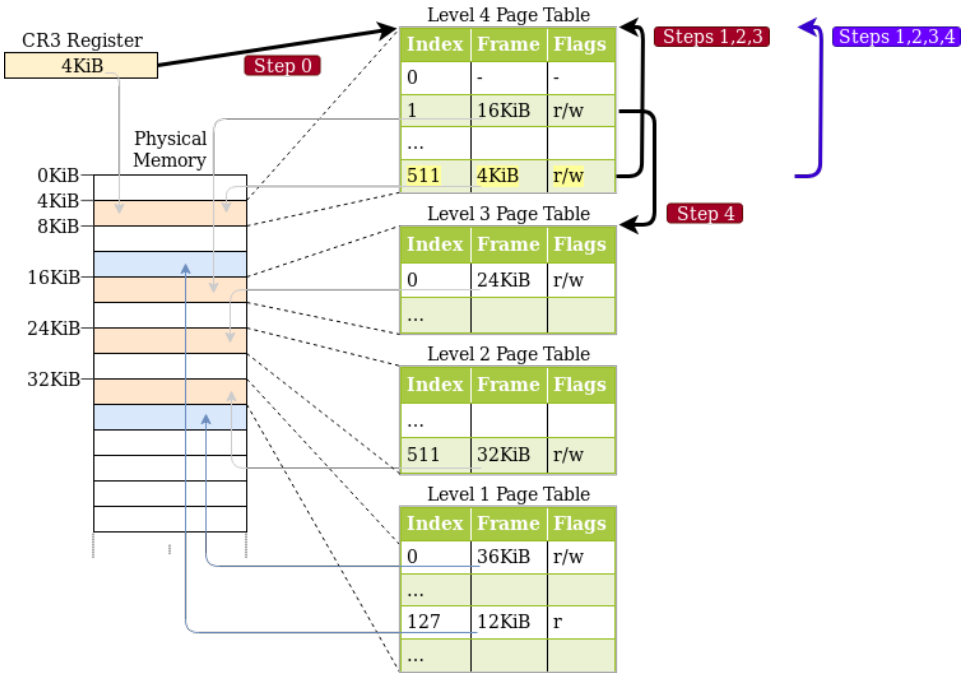

Figura 5.1: Ejemplo para acceder a la página de nivel 3 (en rojo) y a la página de nivel 4 (en azul). Fuente: [15]

virtual, de esta manera se puede acceder mediante las dirección virtual hasta que lleguemos a la última posición, en la cual haríamos un acceso a la posición que buscamos.

Afortunadamente la *crate bootloader* implementa ambas opciones, y tras considerarlo detenidamente se ha decidido utilizar el mapeado directo, ya que aunque hay que usar más espacio lo cual puede ser problemático en sistemas con poca memoria, es mucho más simple que hacerlo de manera recursiva.

Posterior a esto es posible utilizar las páginas para cualquier cosa necesaria, por lo que implementar la paginación permitirá utilizar todo lo relacionado con la librería *alloc* que permite usar estructuras tales como `Box` (un valor en el *heap*) `Vec` (una serie de valores en el *heap*) y `String` (cadenas de caracteres).

Además teniendo un *heap*, es posible simular una memoria secundaria en el *heap* para probar la implementación del sistema de Archivos, simplemente creando una estructura que simule el disco con el contenido de los datos representado por un `Vec<u8>`[1]en el mismo *heap*.

[1]Utilizando u8 para representar una memoria en bytes

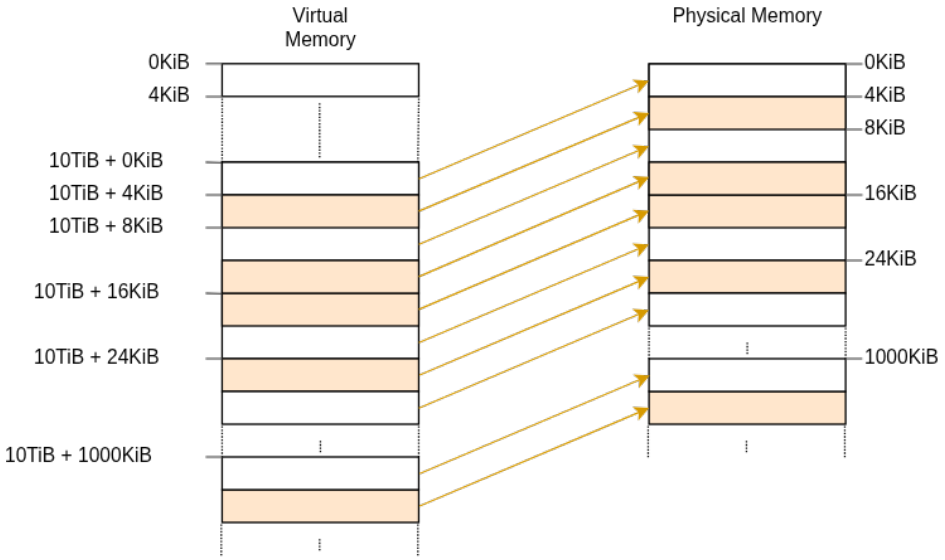

Figura 5.2: Estructura de paginación almacenando todas las posiciones en una dirección muy alejada de la memoria. Fuente: [15]

5.6. Sistema de Archivos

En lo respectivo al sistema de archivos se ha decidido implementar FAT32, con una serie de funciones que permitirán editar los sectores del disco que se decida usar en cada momento, la implementación se ha hecho de manera que no dependa del tipo de disco, sino de dos funciones de un *trait* (algo similar a las interfaces en otros lenguajes de programación) que leen y escriben en sectores de 512 bytes.

El uso de FAT32 sobre otros sistemas de archivos se ha tomado por la sencillez en comparación a otros sistemas de archivos como NTFS o ext2, lo cual permite centrarnos en los aspectos más fundamentales del sistema operativo sin tener que añadir una complejidad excesiva a este componente.

Para implementar el sistema de archivos se ha simulado un disco duro sobre la memoria principal, como se mencionó anteriormente se ha utilizado una estructura *Vec* sobre el heap con un tamaño dinámico, por lo tanto se ha reservado tanta memoria como fuese necesaria para ello.

Esta implementación se ha hecho porque era la más directa después de implementar el heap, en lugar de tener que tratar con los dispositivos de memoria secundaria se ha podido empezar a implementar directamente el sistema de archivos.

En lo relacionado a la memoria masiva, al menos para esta primera versión del sistema operativo no se permite crear más que un dispositivo, por motivos

de simplicidad, aunque es más que suficiente para probar las características que se han implementado hasta el momento.

Adicionalmente a estas funcionalidades se ha implementado el *trait* Display de la librería fmt [2] para las entradas de los directorios, el BPB y el EBR, para poder mostrar los valores de los mismos más fácilmente.

La implementación de *Display* ha sido crucial para los dispositivos, ya que nos permite imprimir (incluso en el puerto serie que se ha conectado a la terminal que llama a *qemu*) los valores de estos componentes sin la necesidad de añadir ninguna lógica adicional independientemente de cómo se use, esto se debe a que `print!` y `serial_print!` implementan ambos el soporte para *fmt*.

En un disco no volátil no sería necesario darle formato a la memoria en cada inicialización, sin embargo debido a que nuestra implementación se ha diseñado sobre el heap es necesario darle formato al BPB y al EBR cada vez que se inicialice el sistema, ya que al funcionar la RAM sobre condensadores, al descargarse los valores se "borran"(técnicamente se corrompen ya que su valor baja hasta un nivel en el que no se puede diferenciar si era un 0 o un 1).

5.7. Terminal

La implementación de la terminal en este sistema, como se ha mencionado antes, se ha hecho mediante un emulador de terminal, esto es en esencia una función a la que se puede acceder para, por ejemplo, cambiar el buffer que contiene los caracteres introducidos, ejecutar la lógica detrás de diferentes comandos...

En definitiva el emulador de terminal se ha diseñado de tal manera que la función principal, que es `execute_command`, lee el buffer de lo que se ha introducido antes de pulsar la tecla de *enter*, y comprueba mediante un *switch* si es un comando que exista, después ejecuta las líneas relacionadas con ese comando y posteriormente limpia el buffer y permite ejecutar comandos nuevos.

Esta implementación no es la común, lo normal es que el sistema operativo sea capaz de comprobar los programas que contiene, y llamarlos en lugar de tenerlos escritos directamente en un archivo, lo cual sería una terminal al uso.

[2] Como se ha explicado anteriormente un *trait* es equiparable a una interfaz en otros lenguajes, por lo tanto definir Display para *fmt* lo que nos va a permitir es que cuando llamemos a macros como print! que usan *fmt* para dar formato a sus mensajes, decidir cómo se va a imprimir nuestra estructura

El hecho de no tener procesos obliga a buscar una manera alternativa de hacerlo, y la estructura mencionada arriba es probablemente la forma más simple de conseguirlo.

5.8. Depuración

Para la depuración se ha elegido crear un *buffer* que contendrá los mensajes de los componentes que estén marcados para depurar, de esta manera el *buffer* puede ser bloqueado para que las escrituras se hagan de manera atómica.

Estos mensajes serán impresos desde los propios componentes que se encargarán de llamar a una función que imprimirá el mensaje de depuración y detendrá la ejecución hasta que la interrupción de teclado detecte la pulsación de la tecla q, que devolverá el control normal del flujo al programa que se estuviera ejecutando.

Estos mensajes estarán definidos en cada uno de los componentes, usando datos de la ejecución actual, o en caso de que se detenga antes de que se ejecute, de la ejecución que viene, estos datos serán enviados al *buffer* de depuración para ser tratados en la función.

Al comenzar a ejecutar esta función se alojará todo lo almacenado en la posición 0xb8000 (que como se mencionó anteriormente es la posición que la VGA lee para poder imprimir los caracteres alojados sobre ella en pantalla) y se sustituirán por el mensaje de depuración.

Capítulo 6

Implementación

Las partes más importantes de la implementación del proyecto están recogidas en este capítulo con la intención de hacer que este complejo sistema de funciones, estructuras y archivos sea más ameno de comprender para el lector.

6.1. Ejecución

Para ejecutar este proyecto hace falta clonarlo del repositorio `https://github.com/Alux6/UPS`. También es necesario tener los paquetes: `llvm` `qemu-system-x86` `gcc` instalados, además es necesario configurar correctamente tango rust como cargo.

A continuación se encuentran los comandos necesarios para ello, es necesario instalar `rustup`.

```
$ rustup install nightly
$ cargo install bootimage
$ rustup component add rust-src llvm-tools-preview
$ cargo +nightly run
```

El propio cargo, el gestor de paquetes de Rust se encargará de descargar los paquetes necesarios, y también lanzará por el usuario una instancia de *qemu* ejecutando el sistema operativo, para que el usuario pueda probar el mismo.

Dentro del repositorio se encuentran los mismos pasos listados anteriormente junto con un conciso manual de uso, que explica de manera simple los comandos más importantes y el uso del sistema.

6.2. Estructura

En el árbol de directorios que se muestra abajo se muestran los archivos y directorios más importantes:

```
/
├── Cargo.toml … En este archivo se guardan todos
                 los metadatos, dependencias y
                 configuraciones de un proyecto en
                 Rust.
├── src/ … El directorio src/ almacena el
           código fuente de la funcionalidad
           en Rust.
├── tests/ … El directorio tests almacena el
             código fuente relacionado a los
             test de integración.
└── x86_64-ups.json … Este archivo es el target
                      personalizado de compilación de
                      nuestro sistema.
```

6.2.1. Cargo.toml

Los archivos de las crate en Rust se escriben en toml (Tom's Obvious Minimal Language), un formato de archivos pensado para ser simple y minimalista, usado principalmente para configuración.

Este archivo en Rust se divide principalmente en tres bloques:

Metadatos del paquete Este bloque del archivo es en el que se definen los metadatos de las crates, en él se definen cosas como los autores, la versión, la edición o el nombre del paquete, si alguien quisiera usar la crate una vez publicada podrá ver todos estos datos para comprobar si está actualizado, si es la versión que buscan, identificar el paquete...

```
[package]
name = "ups"
version = "0.0.0"
authors = ["Alejandro Machuca Jiménez <alux6mc@gmail.com>"]
edition = "2024"
```

Dependencias

En el bloque de dependencias se añaden todos los paquetes que son necesarios para que se ejecute el proyecto junto con la versión que estamos utilizando, ya que por ejemplo en este proyecto se usa la versión 0.9 de *bootloader* ya que en las versiones más recientes ha habido ciertos cambios que hacen que algunas funciones hayan dejado de estar disponibles.

También es posible decidir qué partes del paquete queremos importar, ya que algunos paquetes no se importan por completo por tener funcionalidades que son incompatibles entre ellas.

```
[dependencies]
linked_list_allocator = "0.9.0"
bootloader = {version = "0.9", features = ["map_physical_memory"]}
volatile = "0.2.6"
spin = "0.5.2"
x86_64 = "0.15.2"
uart_16550 = "0.2.0"
pic8259 = "0.10.1"
pc-keyboard = "0.7.0"
```

Configuración

Es el tercer tipo de bloque, en realidad son varias entradas pero dado que son del mismo tipo las hemos englobado en el mismo grupo, estas entradas configuran las diferentes partes de los paquetes, de la ejecución del propio paquete...

```
[package.metadata.bootimage]
test-args = [
    "-device", "isa-debug-exit,iobase=0xf4,iosize=0x04", "-serial", "stdio
        ",
    "-display", "none"
]
run-args = [
        "-serial", "stdio"
]
test-timeout = 300
test-success-exit-code = 33
```

6.2.2. x86_65-ups.json

Dado que no hay acceso a las facilidades que normalmente se tienen al trabajar sobre un sistema operativo, ha sido necesario definir las normas de compilación para Rust, por lo tanto hace falta crear un archivo *target*, que será el que informe al compilador de las características de nuestro supuesto sistema.

```
{
    "llvm-target": "x86_64-unknown-none",
    "data-layout": "e-m:e-p270:32:32-p271:32:32-p272:64:64-i64:64-i128
        :128-f80:128-n8:16:32:64-S128",

    "arch": "x86_64",
    "target-endian": "little",

    "target-pointer-width": "64",
    "target-c-int-width": "32",
```

```
    "os": "none",

    "executables": true,
    "linker-flavor": "ld.lld",
    "linker": "rust-lld",
    "panic-strategy": "abort",
    "disable-redzone": true,
    "features": "-mmx,-sse,+soft-float",
    "rustc-abi": "x86-softfloat"
}
```

A remarcar las especificaciones de *little endian*, la arquitectura y la especificación de que no se usa ningún sistema operativo.

6.2.3. tests

Este directorio contiene el código fuente para los *tests* de integración, es decir, los *tests* que hacen a los componentes interactuar entre ellos, estos archivos serán ejecutados

6.3. Código fuente

En esta sección se encuentra el código de los archivos fuente que conforman este proyecto, se señalará la estructura de cada uno y se incluirán trozos de código de las partes más importantes dentro de cada uno.

6.3.1. main

El programa *main* es el punto de entrada de este proyecto, donde se hacen todas las inicializaciones y lo primero en ejecutarse al inicializar el sistema, en él se encuentran diversas líneas de código interesantes, aunque cabe remarcar tres puntos importantes:

- **[!no_std] [!no_main]**

  Estos dos atributos conllevan diversas implicaciones, aunque las más evidentes van a ser una independencia absoluta de la librería estándar y la inexistencia de una función *main*. Esta desconexión se refleja en que no estarán disponibles ciertas características como la protección contra un desbordamiento de pila o (como es evidente) la librería estándar, aunque se podrán implementar otras que no vienen directamente implementadas como el heap y algunas estructuras.

  Sin embargo no es posible escribir código *firmware*, un kernel o un *bootloader* usando la std, y si tendremos a mano la librería *core* (que será de gran ayuda durante la implementación del sistema operativo). [23]

El hecho de no incluir una función *main* permitirá declarar un punto de entrada para nuestro sistema operativo, lo que significa que se puede definir un nombre mucho más representativo para el punto de entrada, como por ejemplo ***kernel_main***

- **kernel_main (o entry point)**

  En la función kernel *main* es donde se va a definir todas nuestras estructuras básicas, a continuación se explican las líneas más importantes del código.

```
1 interrupts::mask_irq1();
```

En esta línea se desactiva la interrupción de teclado, esto se hace, como se explicó anteriormente, porque una interrupción durante la inicialización podría ser fatal, corromper el sistema y detener la inicialización, por tanto lo que se hace es desactivar la interacción del usuario para evitar que pueda interferir.

Inicialización del sistema

```
1  ups::init();
2  println!("Hello world!");
3
4  let phys_mem_offset = VirtAddr::new(boot_info.physical_memory_offset)
       ;
5  let mut mapper = unsafe { memory::init(phys_mem_offset) };
6  let mut frame_allocator = unsafe {
7      BootInfoFrameAllocator::init(&boot_info.memory_map)
8  };
9
10 allocator::init_heap(&mut mapper, &mut frame_allocator)
11     .expect("heap initialization failed");
12
13 println!("Heap size: {} MB", allocator::HEAP_SIZE / 1024 / 1024);
```

En la línea 1 podemos ver que se llama a `ups::init()`, esta función se explicará en la subsección *lib*, ya que pertenece a ella pero engloba un conjunto de otras inicializaciones que permiten el correcto funcionamiento del sistema.

En las líneas de la 4 a la 8 podemos ver que se crean estructuras básicas para la memoria virtual, básicamente se obtiene la posición de inicio de la tabla (como se explicó en el capítulo de diseño) para poder acceder a los valores de la memoria física, a continuación se crea un mapa que relaciona las posiciones virtuales a las físicas utilizando las posiciones de memoria que hay desde el inicio de la tabla, obtenido en la línea anterior, hasta el final de la misma.

A continuación inicializamos una estructura que se encargará de crear un iterador para obtener los marcos de página que estén libres, de esta manera podremos usarlos, además esta estructura implementará las funciones necesarias para almacenar páginas en los marcos de página, lo cual permitirá guardar datos usando la paginación.

Por último se inicializa el heap en memoria y se comprueba si ha ocurrido algún error, como medida de transparencia adicional se imprime el tamaño del heap alojado.

Creación del sistema de archivos

```
1  {
2      let mut block_device = BLOCK_DEVICE.lock();
3
4      // Create the filesystem using the locked &mut RamDisk
5      let mut fs = FileSystem::new(&mut *block_device).expect("Failed
           to create FS");
6
7      println!("FAT starts at sector {}", fs.fat_start);
8
9      let occupied = fs.count_occupied_clusters();
10     println!("Occupied clusters: {}", occupied);
11
12     fs.init_fats();
13     println!("FAT was just set up");
14
15     let occupied = fs.count_occupied_clusters();
16     println!("Occupied clusters: {}", occupied);
17
18     fs.create_root_dir().unwrap();
19
20     let occupied = fs.count_occupied_clusters();
21     println!("Occupied clusters: {}", occupied);
22
23
24     fs.create_file(2u32,&"Hellowo.rld").unwrap();
25     fs.create_dir(2u32,&"Hellodir").unwrap();
26
27     let occupied = fs.count_occupied_clusters();
28     println!("Occupied clusters: {}", occupied);
29 }
```

Este código como se puede observar está encapsulado entre dos llaves, esto se debe a que en Rust es muy importante el concepto de tiempo de vida de una función.

Cuando hacemos una petición de una posición de memoria y para evitar condiciones de carrera, corrupciones o comportamientos inadecuados, no se permite hacer dos referencias a una misma posición de memoria de manera mutable (O lo que es lo mismo, de manera que se pueda modificar esa posición), por tanto si hacemos una es importante que la liberemos antes de intentar acceder a ella de nuevo.

En este caso el problema está en el sistema de archivos, al acceder a él lo que hacemos es usar posiciones de memoria mutables, lo cual hacemos también dentro del emulador de terminal. Es necesario liberar la memoria antes de llamar al emulador.

Para ello creamos un contexto local, que es para lo que sirven esos corchetes, cuando se llegue al final las variables que no sean estáticas se liberarán igual que pasaría al final de una función con cualquier variable local. De esta manera se hace lo que sea necesario con el sistema de archivos y posteriormente se libera para que se pueda usar de nuevo.

Una vez explicado esto, línea por línea lo que se hace es obtener acceso al dispositivo mediante la función `.lock()` (debido a que el dispositivo es un `mutex`, al pedir un *lock* confirmamos que únicamente habrá una porción de código usándolo en cada momento), este dispositivo de bloques se explicará en profundidad en la subsección ”Sistema de Archivos”.

Posteriormente creamos una estructura de sistema de archivos, que permitirá trabajar sobre los sectores del dispositivo (aunque como ya se ha mencionado, este dispositivo es simulado sobre el heap), mediante esta la instancia de esta estructura es posible llamar a `init_fats`, una función que inicializará las tablas (como se explicó en detalle en capítulos anteriores) y creará así la estructura básica necesaria para el sistema de archivos FAT32.

Posteriormente, en la línea 18 se crea el *cluster root*, que será el punto de partida para cualquier archivo que se quiera crear. Adicionalmente en las líneas 24 y 25 se crean un archivo dentro de este directorio raíz, y un directorio también dentro de este directorio, para tener una estructura básica que demuestre que es posible crear ambos mediante esta implementación.

El resto de líneas lo que hacen es imprimir datos sobre lo que se está haciendo para comprobar que funciona y para comprender qué se está creando, por lo tanto no merecen una explicación en detalle.

Fin de la inicialización

```
disable_hardware_cursor();

shell::init();

interrupts::unmask_irq1();

#[cfg(test)]
test_main();
```

```
9
10 ups::hlt_loop();
```

En las líneas anteriores lo que se hace es llamar a la función disable_hardware_cursor(), una función que en esencia desactiva el cursor de la memoria VGA para que no haga falta preocuparse por él, de esta manera y dado que la posición del cursor es siempre conocida en este sistema (porque está al final de la última línea que se está escribiendo) es posible ganar en simplicidad ignorando este componente.

Posteriormente se llama a la inicialización del emulador de terminal, lo cual se explicará en su debida subsección, y se vuelve a activar la interrupción de teclado, para poder interactuar con la terminal.

Finalmente en la línea 10 se llama a una función auxiliar que se encargará de mantener a la CPU encendida sin ejecutar nada, esto se hace porque aún usando el emulador, si hacemos un bucle infinito en lugar de esta llamada empieza a subir de manera indeterminada el uso de la CPU, mientras que al añadir esto y en reposo, el uso de la CPU (Al menos en el sistema en el que se ha probado) se mantiene por debajo del 2 %, frente al 40 % que se ha llegado a observar en alguna ocasión.

- **Código de testado** Finalmente y como se puede observar, hay varias partes del código que contienen el atributo #[cfg(test)], esto lo que indica es que ese código únicamente será ejecutado en el caso de que llamemos a la ejecución del sistema usando la orden de testado:

```
cargo test
```

En el caso de que ejecutemos de manera normal como se explicó en el apartado *Ejecución* de este mismo capítulo no será ejecutado, esta herramienta lo que permite es definir casos de uso para probar los programas en Rust que podrían ser costosos, peligrosos o incluso erróneos (es posible dar por bueno un test si devuelve un error) que durante el funcionamiento normal del programa nunca sucederían, lo cual permite desarrollar nuestros *tests* a la par que nuestros programas.

6.3.2. lib

El archivo *lib* recoge varias de las funciones de carácter general que se utilizan en este proyecto, es el archivo en el que se define todo lo relacionado con la ejecución de *tests*, la inicialización básica del sistema y algunas funciones necesarias más.

En este archivo, como se hizo en el *main*, especificamos que no se va a utilizar

la *libreria* estándar, también se especifica que el *kernel main* se encuentra en este archivo y que lo relacionado con los *tests*, como la lógica detrás de los *panic* para los *tests*, o el código que va a terminar la ejecución del emulador tras finalizar el test.

init

```
1 pub fn init() {
2     gdt::init();
3     interrupts::init_idt();
4     unsafe { interrupts::PICS.lock().initialize() };
5     x86_64::instructions::interrupts::enable();
6 }
```

Como podemos ver, el *init* de este archivo llama a varias funciones de inicialización diferentes, la línea 2 llama a la inicialización de la GDT (Global Descriptor Table) que debe ser la primera inicialización que se haga, ya que es necesaria para hacer la inicialización de las interrupciones, que viene justo después.

La última línea, la línea 5, habilita todas las interrupciones, lo cual podría parecer contradictorio con lo que hicimos en el *main* al desactivar la interrupción de teclado, sin embargo tiene mucho sentido, ya que lo que hemos hecho ha sido desactivar la señal interrupción (lo que se conoce como enmascarar la línea `IRQ1`), no la interrupción en si, por tanto la interrupción estará activada, pero el *PIC* nunca llegará a enviar esa señal.

La inicialización de las interrupciones será explicada en el apartado del archivo *interrupts*.

str_to_fat_name

```
pub fn str_to_fat_name(filename: &str) -> [u8; 11] {
    let mut fat_name = [b' '; 11];

    let parts: Vec<&str> = filename.split('.').collect();

    if let Some(name) = parts.get(0) {
        for (i, &b) in name.as_bytes().iter().take(8).enumerate() {
            fat_name[i] = b.to_ascii_uppercase();
        }
    }

    if parts.len() > 1 {
        if let Some(ext) = parts.get(1) {
            for (i, &b) in ext.as_bytes().iter().take(3).enumerate() {
                fat_name[8 + i] = b.to_ascii_uppercase();
            }
        }
    }

```

```
    fat_name
}
```

Testable

```
pub trait Testable {
    fn run(&self) -> ();
}

impl<T> Testable for T
where
    T: Fn(),
{
    fn run(&self) {
        serial_print!("{}...\t", core::any::type_name::<T>());
        self();
        serial_println!("{}",Green("[ok]"));
    }
}
```

La definición del *trait* `Testable` para cualquier función lo que permite es definir, para cualquier función, lo que va a pasar, al convertir una función en un test lo único que hace falta es ejecutar esa función con `nombre.run()`.

Como hemos definido al implementar el *trait* `Testable`, al ejecutar `run` se imprimirá el nombre de la función, después la función será ejecutada y a continuación se imprimirá un mensaje indicando que la función se ha ejecutado de manera satisfactoria si fuera el caso, y un mensaje de error si el programa ha fallado en algún momento.

test_runner

```
pub fn test_runner(tests: &[&dyn Testable]) {
    serial_println!("Running {} tests", tests.len());
    for test in tests {
        test.run();
    }
    exit_qemu(QemuExitCode::Success);
}
```

Para ejecutar automáticamente las funciones de testado lo que se hace es escribir el atributo `#[test_case]`, que se buscará en todos los archivos para introducirlos en una lista que se le mandará al `test_runner`, que ejecutará los *tests* y posteriormente terminará *qemu*.

test_kernel_main

```
#[cfg(test)]
#[unsafe(no_mangle)]
```

```
fn test_kernel_main(_boot_info: &'static BootInfo) -> ! {
    init();
    test_main();
    hlt_loop();
}
```

En esta función se inicializa el sistema y posteriormente se ejecuta la lógica que permitirá ejecutar todos los *tests* como se ha mencionado anteriormente, puesto que hemos definido esta función como el *entrypoint* en caso de que se esté ejecutando el comando con el argumento test, será lo primero que se ejecute.

test_breakpoint_exception

```
#[test_case]
fn test_breakpoint_exception() {
    x86_64::instructions::interrupts::int3();
}
```

Como muestra de un test unitario, este es el formato que toma el test de llamada a la interrupción de breakpoint.

exit_qemu

```
pub fn exit_qemu(exit_code: QemuExitCode) {
    use x86_64::instructions::port::Port;

    unsafe {
        let mut port = Port::new(0xf4);
        port.write(exit_code as u32);
    }
}
```

En esta función lo que se hace es crear un punto de salida mediante el puerto 0xf4 y mandar el código de salida de Qemu que queramos, estos códigos están en el enumerado llamado QuemuExitCode, donde se han definido las señales relacionadas con Qemu que identifican un código de finalización satisfactorio o fallido.

6.3.3. vga_buffer

Este archivo es en el que se ha definido el driver de video que interactúa con la memoria de vídeo de la VGA, se han definido diversas estructuras simples que forman parte de las estructuras más complejas que se utilizan después para interactuar desde un nivel más alto y de manera segura con la memoria de la VGA.

La razón de ser de este archivo, además de la simplicidad y proveer una capa de abstracción para evitar tener que repetir código todo el rato, es el hecho de dar una herramienta que sea capaz de acceder de manera insegura a la memoria de vídeo, pero que provea al usuario de una manera de acceder de manera segura a la modificación de la misma.

Al escribir este driver se han encapsulado los accesos inseguros en unas breves líneas de código para definir métodos seguros que modifican la memoria de una manera que no tengamos que preocuparnos por la seguridad.

Writer

```
pub struct Writer {
    column_position: usize,
    color_code: ColorCode,
    buffer: &'static mut Buffer,
}
```

La estructura Writer es una estructura que contiene la posición de la columna en la que se está escribiendo, una referencia al buffer (básicamente la dirección de la memoria de la VGA) y un código de color, que define el estilo de los caracteres que se imprimen (en este caso amarillo y el fondo negro).

Es importante comentar también el uso de `volatile`, la función `volatile` se usa para evitar que los compiladores optimicen estas posiciones, debido a que se van a utilizar pero únicamente de manera indirecta (Ya que no se guarda el valor después de escribir), el compilador puede pensar que no se usa en absoluto y quizás en una versión más agresiva del compilador en el futuro, el programa deje de funcionar, al usar `volatile` evitamos que se trate de manera normal, evitando la optimización.

Estos datos son más que suficientes para lo que queremos implementar, las funciones que debemos implementar para esta estructura son:

Escritura de byte

```
pub fn write_byte(&mut self, byte: u8) {
    match byte {
        b'\n' => self.new_line(),
        byte => {
            if self.column_position >= BUFFER_WIDTH {
                self.new_line();
            }

            let row = BUFFER_HEIGHT - 1;
            let col = self.column_position;

            let color_code = self.color_code;
            self.buffer.chars[row][col].write(ScreenChar {
```

```
                ascii_character: byte,
                color_code,
            });
            self.column_position += 1;
        }
    }
}
```

En esta función se define la lógica básica para la escritura de un byte en la memoria, dependiendo del valor del byte, se llamará al método encargado de desplazar las líneas hacia arriba en la pantalla para poder seguir escribiendo en la línea más inferior (esto se conoce como *scrolling*).

En caso de ser cualquier otro carácter se comprueba si la posición desbordaría el *buffer* y lo que se hace es ejecutar la misma lógica mencionada anteriormente, y posteriormente se escribe el byte en la posición correspondiente de memoria y se incrementa la columna del cursor para hacer que avance una posición.

Escritura de una cadena

```
pub fn write_string(&mut self, s: &str) {
    for byte in s.bytes() {
        match byte {
            0x20..=0x7e | b'\n' => self.write_byte(byte),

            _ => self.write_byte(0xfe),
        }
    }
}
```

La función para la escritura de una cadena se hace llamando a la función de escritura de un byte que se ha creado anteriormente, en caso de ser un carácter que no es posible imprimir con la fuente por defecto de la VGA se imprimirá el carácter 0xfe (un carácter con todos los pixeles activados).

Borrado de un byte

```
pub fn delete_byte(&mut self) {
    if self.column_position != 0 {

        self.column_position -= 1;

        let row = BUFFER_HEIGHT - 1;
        let col = self.column_position;

        let blank = ScreenChar {
            ascii_character: b' ',
            color_code: self.color_code,
        };

        self.buffer.chars[row][col].write(blank);
    }
```

```
}
```

La implementación de esta función cubre la necesidad de retroceder en la pantalla, cuando se quiera borrar un carácter que ha sido introducido por el motivo que sea.

La implementación es sencilla, se reduce la posición del cursor y se imprime sobre ella un carácter en blanco, que borrará el valor que hubiera en esa posición de la memoria de video anteriormente, lo cual para el usuario se percibirá como un borrado.

Salto de línea

```
fn new_line(&mut self) {
    for row in 1..BUFFER_HEIGHT {
        for col in 0..BUFFER_WIDTH {
            let character = self.buffer.chars[row][col].read();
            self.buffer.chars[row - 1][col].write(character);
        }
    }
    self.clear_row(BUFFER_HEIGHT - 1);
    self.column_position = 0;
}
```

En el código anterior se muestra cómo se tratan los saltos de línea, básicamente lo que se hace es copiar todas las líneas una línea arriba (excepto la superior para evitar escribir fuera de las posiciones reservadas) y posteriormente borra la línea más inferior, de esta manera dejamos la línea en la que escribimos libre para poder escribir de nuevo, mientras que se desplaza el resto de la información superiormente para no perder estos datos.

Borrado de línea

```
fn clear_row(&mut self, row: usize) {
    let blank = ScreenChar {
        ascii_character: b' ',
        color_code: self.color_code,
    };
    for col in 0..BUFFER_WIDTH {
        self.buffer.chars[row][col].write(blank);
    }
}
```

En esta segunda parte del código se ve el borrado de una línea, lo único que se hace es recorrer la línea que se quiere borrar sustituyendo el valor que hubiera por un carácter vacío.

WRITER

```
lazy_static! {
    pub static ref WRITER: Mutex<Writer> = Mutex::new(Writer {
        column_position: 0,
        color_code: ColorCode::new(Color::Yellow, Color::Black),
        buffer: unsafe { &mut *(0xb8000 as *mut Buffer) },
    });
}
```

Para permitir el uso de esta estructura de manera global lo que se hace es crear una variable estática. Una variable estática tiene un tiempo de vida global, es decir, persiste durante toda la ejecución del programa. Sin embargo, las variables estáticas en Rust deben ser inicializadas en tiempo de compilación, lo cual limita su uso cuando se requiere una inicialización en tiempo de ejecución.

Para resolver este problema, se puede usar la macro `lazy_static!`, que permite definir variables estáticas que se inicializan de forma diferida, es decir, la primera vez que se accede a ellas. Esto es útil, por ejemplo, cuando se quiere definir un `mutex` global que necesita ser inicializado con una función en tiempo de ejecución.

Lazy static significa que la variable únicamente será definida la primera vez que se utilice, por ejemplo para un `Mutex` será definida la primera vez que se llame sobre ella el método .lock(), por tanto de esta manera podemos crear variables globales inicializadas en tiempo de ejecución.

Hoy en día se recomienda utilizar alternativas como `one_cell::sync::Lazy` o `std::sync::OnceLock` (en este caso no es viable usar la std) que se consideran más aceptadas, por motivos de simplicidad y continuidad con el tutorial que se utilizó de base se usará *lazy static* en lugar de estas alternativas.

_print

```
#[doc(hidden)]
pub fn _print(args: fmt::Arguments) {
    use core::fmt::Write;
    use x86_64::instructions::interrupts;

    interrupts::without_interrupts(|| {
        WRITER.lock().write_fmt(args).unwrap();
    });
}
```

En Rust para la impresión por pantalla (de manera similar al `printf` en C) se utilizan macros, específicamente `print!` y `println!`, sin embargo estas macros dependen de la std y de los *drivers* de pantalla que se estén utilizando, por lo tanto como se han definido *drivers* nuevos es necesario reescribir la

implementación de estas macros.

La base de estas macros es una función `print`, a la cual se llamará desde las macros para imprimir por pantalla usando el *writer* que se ha definido con anterioridad, de esta manera se encapsula el uso del *writer* a una única función y podemos implementar el uso de *fmt* (que permitirá dar formato y declarar argumentos para nuestras macros).

En la función `print` podemos ver que se detienen las interrupciones durante la escritura, y se importa la función `write` de la librería *fmt*, lo cual permitirá llamar a `write_fmt()` que dará formato por nosotros a las cadenas que se van a escribir sobre el *writer*. Se escribe sobre un `lock` del *writer* (que permite controlar el acceso al *writer*) para que se haga de manera atómica.

macros

```
#[macro_export]
macro_rules! print {
    ($($arg:tt)*) => ($crate::vga_buffer::_print(format_args!($($arg)*)));
}

#[macro_export]
macro_rules! println {
    () => ($crate::print!("\n"));
    ($($arg:tt)*) => ($crate::print!("{}\n", format_args!($($arg)*)));
}
```

La definición de macros en Rust es un tema completamente aparte, muy diferente de la programación usual del lenguaje y en el que no se entrará en este trabajo, sin embargo no hace falta comprender completamente cómo funcionan las macros para comprender lo que está ocurriendo.

En la macro `print!` se está llamando a la función `_print` que se definió anteriormente, junto con una serie de argumentos, mientras que en la macro `println!` se llama a la macro `print!` pero finalizando la cadena con un salto de línea.

test_println_output

```
#[test_case]
fn test_println_output() {
    use core::fmt::Write;
    use x86_64::instructions::interrupts;

    let s = "Some test string that fits on a single line";
    interrupts::without_interrupts(|| {
        let mut writer = WRITER.lock();
        writeln!(writer, "\n{}", s).expect("writeln failed");
        for (i, c) in s.chars().enumerate() {
            let screen_char = writer.buffer.chars[BUFFER_HEIGHT - 2][i].
                read();
```

```
12                 assert_eq!(char::from(screen_char.ascii_character), c);
13             }
14         });
15 }
```

En este test unitario vemos que se llama al *writer*, el cual imprimirá una cadena sobre la memoria y posteriormente comprueba que el valor que hay alojado en memoria es el valor que se ha ordenado imprimir, esto permite comprobar cosas como que se está respetando el *little endian* y que se escribe de manera correcta sobre memoria.

6.3.4. interrupts

Las interrupciones de este sistema se han manejado de manera similar al controlador de teclado, se ha creado un elemento estático para la IDT y se han enlazado en él las interrupciones que se quieren manejar con las funciones que van a manejarlas.

Cada una de las interrupciones tiene una lógica diferente, que se verá en cómo funcionan los handlers, aunque todas coinciden en que su posición en memoria será almacenada sobre la tabla que hemos mencionado previamente.

IDT

```
1  lazy_static! {
2      static ref IDT: InterruptDescriptorTable = {
3          let mut idt = InterruptDescriptorTable::new();
4
5          idt.breakpoint.set_handler_fn(breakpoint_handler);
6          idt.page_fault.set_handler_fn(page_fault_handler);
7          unsafe {
8              idt.double_fault.set_handler_fn(double_fault_handler)
9                  .set_stack_index(gdt::DOUBLE_FAULT_IST_INDEX);
10         }
11         idt[InterruptIndex::Timer.as_u8()]
12             .set_handler_fn(timer_interrupt_handler);
13         idt[InterruptIndex::Keyboard.as_u8()]
14             .set_handler_fn(keyboard_interrupt_handler);
15         idt
16     };
17 }
```

Esta tabla usa como base de su estructura la librería x86_64 que implementa para los desarrolladores cosas tales como estructuras de esta arquitectura sin hacer uso de la std, lo cual significa que podremos usar esta librería sin necesidad de tener acceso a la std.

Como podemos ver se asocian los handler con las interrupciones utilizando el nombre de la interrupción (o su índice en la PIC) y sobre esto, la función

set_handler_fn y el nombre de la función que se va a encargar de ello.

Handlers

En la siguiente lista se muestra la implementación de todos los *handlers* de las interrupciones.

- **Excepción de fallo de página**

```
extern "x86-interrupt" fn page_fault_handler(
    stack_frame: InterruptStackFrame,
    error_code: PageFaultErrorCode,
) {
    use x86_64::registers::control::Cr2;

    println!("EXCEPTION: PAGE FAULT");
    println!("Accessed Address: {:?}", Cr2::read());
    println!("Error Code: {:?}", error_code);
    println!("{:#?}", stack_frame);
    hlt_loop();
}
```

  La excepción de fallo de página como se explicó en capítulos anteriores, salta cuando se intenta acceder a un marco de página que no ha sido alojado.

  El handler de esta excepción imprime un mensaje informando del fallo, y de una serie de datos que permitirán saber qué marco ha dado el error, posteriormente llama a la función que se mencionó previamente para evitar que la CPU aloje una gran cantidad del uso de la CPU.

- **Interrupción de teclado** Debido a que esta función es demasiado grande, se dividirá en varias partes para poder explicarla de manera estética.

  Inicialización de la función

```
extern "x86-interrupt" fn keyboard_interrupt_handler(
    _stack_frame: InterruptStackFrame)
{
    use pc_keyboard::{layouts, DecodedKey, HandleControl, Keyboard,
        ScancodeSet1};
    use spin::Mutex;
    use x86_64::instructions::port::Port;

    lazy_static! {
        static ref KEYBOARD: Mutex<Keyboard<layouts::Us104Key,
            ScancodeSet1>> =
        Mutex::new(Keyboard::new(ScancodeSet1::new(),
            layouts::Us104Key, HandleControl::Ignore)
        );
    }

    let mut port = Port::new(0x60);
```

```
16     let mut keyboard = KEYBOARD.lock();
```

En la primera parte de la función lo que se hace es importar las diversas librerías que van a usarse, entre ellas podemos ver `mutex`, que se usará para evitar que se acceda al teclado de manera concurrida, para evitar condiciones de carrera.

También se puede ver la instanciación del puerto `0x60`, puerto que recibe los datos de las pulsaciones de teclado, junto a la llamada a `lock` sobre `keyboard` que devolverá una instancia de esta estructura que permitirá usarla.

Puede parecer raro que se esté llamando a `lock` porque parecería que `keyboard` se está creando en esta misma función, sin embargo esto sería un error, ya que se crea como estática, por lo tanto aunque realmente se está inicializando la primera vez que se pulsa una tecla, esta función persistirá hasta que se detenga el sistema operativo.

Tratamiento de la pulsación

```
1  let scancode: u8 = unsafe {port.read() };
2  if let Ok(Some(key_event)) = keyboard.add_byte(scancode) {
3      if let Some(decoded) = keyboard.process_keyevent(key_event) {
4          if let DecodedKey::Unicode(character) = decoded {
5              let byte = character as u8;
6              if DEBUG_MODE.load(SeqCst){
7                  if character == 'q' {
8                      DEBUG_MODE.store(false, SeqCst);
9                  }
10             }
11             else {
12                 let mut term = TERMINAL.lock();
13                 if character == '\u{8}' {
14                     term.pop_char();
15                     WRITER.lock().delete_byte();
16                 }
17                 else{
18                     let mut result: String = term.push_char(byte);
19                     if character == '\n' || character == '\r' {
20                         EXECUTE_COMMAND.store(true, SeqCst);
21                     }
22                         print!("{}", result);
23                 }
24             }
25         }
26     }
27 }
```

En esta segunda porción del código leemos el puerto y se hacen una serie de comprobaciones para ver si el puerto tenía datos válidos, si lo que se a pulsado era un carácter, posteriormente si el carácter que era y se puede remarcar la comprobación del DEBUG_MODE.

Este modo se explicará en profundidad en el apartado *debug* de este capítulo, pero de manera simplificada es un modo en el que únicamente aceptamos que se pulse la tecla `q` para abandonar el modo y volver al buffer original.

Por otra parte si no nos encontrásemos en el modo *debug*, lo que se hace es comprobar si hemos pulsado la tecla borrar, en cuyo caso borramos tanto en la terminal como en el *writer* del driver de video.

En otro caso comprobamos si hemos pulsado *intro* en cuyo caso se modifica la flag del modo de ejecución, esta flag se utilizará desde el bucle principal para ordenar a la terminal que ejecute el comando que haya reservado en el buffer, o cualquier otra cosa para imprimirlo.

Fin de la interrupción

```
    unsafe {
        PICS.lock()
            .notify_end_of_interrupt(InterruptIndex::Keyboard.as_u8()
                );
    }
}
```

Las interrupciones, a diferencia de las excepciones, deben informar de su final, por lo tanto es necesario reservar el uso del PIC para informar del fin de la interrupción.

- **Interrupción de reloj**

```
extern "x86-interrupt" fn timer_interrupt_handler(
    _stack_frame: InterruptStackFrame)
{
    unsafe {
        PICS.lock()
            .notify_end_of_interrupt(InterruptIndex::Timer.as_u8());
    }
}
```

La interrupción de reloj únicamente devuelve el final de la interrupción, por lo que no bloquea el funcionamiento pero tampoco genera una excepción doble, ya que se considera que ha sido manejada.

- **Excepción de breakpoint**

```
extern "x86-interrupt" fn breakpoint_handler(
    stack_frame: InterruptStackFrame)
{
    return;
}
```

La excepción de breakpoint, no hace nada, se ha diseñado para implementarla en un futuro sin que interfiera con el funcionamiento de la CPU.

- **Excepción doble**

```
extern "x86-interrupt"   JK,.L–LÑ fn double_fault_handler(
    stack_frame: InterruptStackFrame, _error_code: u64) -> !
{
    panic!("EXCEPTION: DOUBLE FAULT\n{:#?}", stack_frame);
}
```

Esta excepción únicamente provoca un *panic*, lo que quiere decir que el proceso se detendrá e imprimirá el mensaje que contiene la llamada, junto con la información sobre la pila que hubiera en el momento en que se lanzó.

6.3.5. memory

En esta parte del código fuente se define la paginación, el acceso a los marcos de memoria y las estructuras del iterador de marcos de página y la lógica de las páginas.

Marcos de páginas

```
pub struct BootInfoFrameAllocator {
    memory_map: &'static MemoryMap,
    next: usize,
}

impl BootInfoFrameAllocator {

    pub unsafe fn init(memory_map: &'static MemoryMap) -> Self {
        BootInfoFrameAllocator {
            memory_map,
            next: 0,
        }
    }

    fn usable_frames(&self) -> impl Iterator<Item = PhysFrame> {
        let regions = self.memory_map.iter();
        let usable_regions = regions
            .filter(|r| r.region_type == MemoryRegionType::Usable);

        let addr_ranges = usable_regions
            .map(|r| r.range.start_addr()..r.range.end_addr());

        let frame_addresses = addr_ranges.flat_map(|r| r.step_by(4096));

        frame_addresses.map(|addr| PhysFrame::containing_address(PhysAddr
            ::new(addr)))
    }
}
```

La lógica de los marcos de páginas es bastante sencilla, únicamente creamos una estructura que contenga el mapa de memoria y la siguiente posición de las páginas.

Escribimos como se puede ver en la línea 15, una función que va a devolver un iterador sobre la memoria física que itera sobre las regiones disponibles para alojar.

Esto se hace filtrando unicamente las regiones marcadas como disponibles, después se trasforman en rangos de direcciones y después se avanza sobre ellas en incrementos de 4096 bytes (4KiB) ya que este es el tamaño de una página en el sistema, finalmente convierte estas direcciones en la estructura PhysFrame, que representa un marco de página físico.

Alojador de marcos

```
unsafe impl FrameAllocator<Size4KiB> for BootInfoFrameAllocator {
    fn allocate_frame(&mut self) -> Option<PhysFrame> {
        let frame = self.usable_frames().nth(self.next);
        self.next += 1;
        frame
    }
}
```

Para ser capaces de alojar posiciones de memoria se implementa un allocator, que se encargará de alojar los valores en la memoria física. Esto se hace obteniendo el *frame* que se ha marcado como siguiente disponible y se aumenta el valor del siguiente marco, finalmente se devuelve el marco que se ha obtenido en la primera línea.

Aunque este código directamente no es el que aoja la memoria, es el que devolverá los marcos con el formato necesario para que se pueda alojar por parte del módulo allocator.

inicialización

```
pub unsafe fn init(physical_memory_offset: VirtAddr) -> OffsetPageTable<'
    static> {
    unsafe {
        let level_4_table = active_level_4_table(physical_memory_offset);
        OffsetPageTable::new(level_4_table, physical_memory_offset)
    }
}
```

La inicialización de la memoria para hacer uso de la paginación se compone de dos partes, la primera en la que se crea la tabla de nivel 4 necesaria en base al offset (es decir, ese mapeo directo que se hizo en una posición alejada de memoria para poder accederlo posteriormente) y la segunda parte, en la

que devolvemos la estructura del mapa que toma como argumentos la tabla de nivel 4 y el desplazamiento en memoria física.

Creación de la tabla de nivel 4

```
1  pub unsafe fn active_level_4_table(physical_memory_offset: VirtAddr)
2  -> &'static mut PageTable
3  {
4      use x86_64::registers::control::Cr3;
5      let (level_4_table_frame, _) = Cr3::read();
6      let phys = level_4_table_frame.start_address();
7      let virt = physical_memory_offset + phys.as_u64();
8      let page_table_ptr: *mut PageTable = virt.as_mut_ptr();
9
10     unsafe { &mut *page_table_ptr }
11 }
```

En la línea 4 se obtiene el marco físico donde reside la tabla de nivel 4, en las líneas 6 y 7 se calcula la posición de la tabla en memoria virtual donde hemos mapeado la memoria física a la virtual. Posteriormente convertimos esta posición en un puntero y lo devolvemos.

6.3.6. allocator

```
pub fn init_heap(
    mapper: &mut impl Mapper<Size4KiB>,
    frame_allocator: &mut impl FrameAllocator<Size4KiB>,
) -> Result<(), MapToError<Size4KiB>> {
    let page_range = {
        let heap_start = VirtAddr::new(HEAP_START as u64);
        let heap_end = heap_start + HEAP_SIZE as u64 - 1u64;
        let heap_start_page: Page = Page::containing_address(heap_start);
        let heap_end_page: Page = Page::containing_address(heap_end);
        Page::range_inclusive(heap_start_page, heap_end_page)
    };

    for page in page_range {
        let frame = frame_allocator
            .allocate_frame()
            .ok_or(MapToError::FrameAllocationFailed)?;
        let flags = PageTableFlags::PRESENT | PageTableFlags::WRITABLE;
        unsafe {
            mapper.map_to(page, frame, flags, frame_allocator)?.flush()
        };
    }

    unsafe {
        ALLOCATOR.lock().init(HEAP_START, HEAP_SIZE);
    }

    Ok(())
}
```

El allocator es necesario para poder almacenar estructuras como `Vec` o `String`, ya que estas estructuras se almacenan en el heap, por tanto es

razonable asumir que, dado que el heap es una estructura que se almacena en páginas y que hace uso de la paginación, será necesario utilizar las funciones y estructuras que se definieron en el apartado de paginación.

Nuestro allocator consiste de dos partes principales, una función de inicialización y la configuración de nuestro allocator personalizado como el allocator global.

En el código anterior podemos ver que nuestra función de inicialización hace uso de dos argumentos, el `mapper` y el `frame allocator`, que serán utilizados para alojar todas las páginas necesarias. Al final de nuestra función después de alojar todas las páginas del heap se inicializa el allocator con los valores de inicio y final del heap.

```
#[global_allocator]
static ALLOCATOR: LockedHeap = LockedHeap::empty();
```

Con esta parte del código esencialmente lo que se hace es definir como allocator global el allocator que se acaba de definir, de esta manera desde el momento en que se ejecuta la función `init_heap` es posible usar estructuras, librerías y técnicas de programación que hagan uso del heap, dado que ahora hemos definido la capa de abstracción necesaria para usarlo.

6.3.7. shell

Para este módulo se emplean los mismos métodos que anteriormente, crear una estructura con diversas funciones que nos permitan interactuar con ella, inicializar una instancia de la misma de con *lazy static* y hacerla pública para que sea accesible desde cualquier módulo.

Estructura de la terminal

```
pub struct Terminal {
    cwd: u32,
    buffer: [u8; 512],
    index: usize,
}
```

Como podemos observar, la estructura de la terminal es bastante sencilla, simplemente se guarda en la estructura la id del clúster en el que se encuentra, para poder interactuar con el sistema de archivos desde la terminal. También contiene un buffer que será donde se guarden los comandos que posteriormente vamos a ejecutar, y un índice para saber en qué posición del buffer se encuentra el cursor.

Métodos de la terminal

push char

```
pub fn push_char(&mut self, c: u8) -> String {
    let mut out = String::new();
    if self.index < self.buffer.len() {
        self.buffer[self.index] = c;
        self.index += 1;
    }
    let _ = write!(out, "{}", c as char);
    out
}
```

Para añadir un carácter al buffer se crea una nueva cadena, a continuación si aún queda espacio para añadir un carácter se añade al final del buffer y se devuelve como cadena el valor que se había introducido.

execute command

```
pub fn execute_command(&mut self) -> String {

    let mut out = String::new();

    let _ = writeln!(out,"");

    let input = &self.buffer[..self.index];
    if let Ok(cmd) = core::str::from_utf8(input) {

        let trimmed = cmd.trim();
        let mut parts = trimmed.split_whitespace();

        let command = parts.next().unwrap_or("");
        let arg = parts.next();

        match command {
            ...
        }

    } else {
        let _ = writeln!(out, "Invalid UTF-8 input");
    }

    let _ = write!(out, "> ");

    self.index = 0;
    out
}
```

En esta sección del código podemos comprobar la lógica que se sigue a la hora de ejecutar un comando, se lee lo que contiene el buffer, se divide en partes y se accede a un `match` (similar a un `switch` en C) para decidir qué comando se va a ejecutar.

Como se explicó anteriormente no se ha implementado la lógica relacionada

a procesos y programas, por lo tanto no es posible hacer una terminal real que acceda a un directorio y ejecute comandos de los archivos que contiene, por lo tanto se ha optado por implementar comandos como opciones dentro de un `switch`.

Después de la ejecución de la lógica de los comandos se añade al buffer de salida (ya que la función devuelve una cadena, `out`) se devuelve lo que haya decidido la función, lo cual permite decidir cómo se quiere tratar la cadena que devuelve esta función.

pop char

```
pub fn pop_char(&mut self){
    if self.index > 0 {
        self.index -= 1;
    }
    self.buffer[self.index] = b' ';
}
```

Para borrar caracteres del buffer se retrocede en uno la posición en la que estábamos escribiendo y se sustituye por un valor en blanco.

6.3.8. serial

Para poder hacer la ejecución de los *tests* sin tener que abrir el emulador en modo gráfico cada vez es posible ejecutar el emulador sin abrir una ventana, para agilizar el proceso, además de poder imprimir al puerto serie, puerto que se puede relacionar desde el emulador hacia la terminal que lo ejecuta, de esta manera podemos imprimir mensajes con el resultado de los *tests*.

Macros

```
lazy_static! {
    pub static ref SERIAL1: Mutex<SerialPort> = {
        let mut serial_port = unsafe { SerialPort::new(0x3F8) };
        serial_port.init();
        Mutex::new(serial_port)
    };
}

#[doc(hidden)]
pub fn _print(args: ::core::fmt::Arguments) {
    use core::fmt::Write;
    use x86_64::instructions::interrupts;

    interrupts::without_interrupts(|| {
        SERIAL1
            .lock()
            .write_fmt(args)
            .expect("Printing to serial failed");
    });
}
```

```

#[macro_export]
macro_rules! serial_print {
    ($($arg:tt)*) => {
        $crate::serial::_print(format_args!($($arg)*));
    };
}

#[macro_export]
macro_rules! serial_println {
    () => ($crate::serial_print!("\n"));
    ($fmt:expr) => ($crate::serial_print!(concat!($fmt, "\n")));
    ($fmt:expr, $($arg:tt)*) => ($crate::serial_print!(
        concat!($fmt, "\n"), $($arg)*));
}
```

Al inicio de este código se crea un elemento estático que representará el puerto serie, posteriormente se inicializa y se crea un `mutex` sobre él, de esta manera no se podrá accederse concurrentemente a la posición de la memoria, acceso que debe ser `unsafe` (inseguro) ya que Rust es incapaz de asegurar un acceso seguro a través de un puntero a una posición de memoria.

Podemos observar que de igual manera a como se hizo con la macro del controlador de vídeo se definirán dos macros, una básica y otra que introduce un salto de línea al final, estas macros serán utilizadas para comunicarnos a través del puerto serie.

Códigos de color

```
pub struct Red(pub &'static str);
pub struct Green(pub &'static str);

impl fmt::Display for Red {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(f, "\x1B[30m")?;
        write!(f, "{}", self.0)?;
        write!(f, "\x1B[0m")?;
        Ok(())
    }
}

impl fmt::Display for Green {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(f, "\x1B[32m")?;
        write!(f, "{}", self.0)?;
        write!(f, "\x1B[0m")?;
        Ok(())
    }
}
```

En este mismo archivo se han definido dos estructuras que permitirán de una manera visual y simple definir el color del texto, actualmente se han implementado únicamente los colores rojo y verde, ya que eran especialmente

útiles para los *tests* (verde cuando el resultado es satisfactorio y rojo cuando no lo es) pero la estructura podría ser utilizada para conformar cualquier color deseado.

La manera de hacerlo es utilizando los códigos *ansi*, que permiten definir el color, en esencia escribir sobre `f`, que es el *formatter*, el valor del color en *ansi*, después se escribe el texto recibido y después el código para escribir de nuevo el color blanco, en caso de que se quisiera por ejemplo cambiar el color de impresión general, únicamente sería necesario cambiar la última línea por una referencia a la variable donde esté guardado este color.

6.3.9. debug

La razón de ser de este archivo es la de almacenar todo lo relacionado con la depuración, es decir, la lógica que hace que este sistema operativo se pueda pausar en ciertos componentes y que los componentes puedan devolver información sobre su funcionamiento entre otras.

El archivo contiene dos estructuras de memoria, SAVED_SCREEN que es la estructura sobre la que vamos a plasmar los datos de la memoria de vídeo de manera auxiliar para que, cuando el usuario desee volver al contexto normal de ejecución, se pueda imprimir de vuelta los datos que había antes de entrar en el contexto de depurado.

DEBUG_BUFFER, el buffer sobre el que los componentes van a escribir sus mensajes de depuración para poder imprimirlos posteriormente en pantalla.

Ambas estructuras se controlan mediante un `Mutex`, para evitar que se accedan de manera concurrente y que puedan producirse condiciones de carrera.

Buffer de depurado

```
1  pub struct DebugBuffer {
2      data: [u8; DEBUG_BUF_SIZE],
3      len: usize,
4  }
5  impl DebugBuffer {
6      pub const fn new() -> Self {
7          Self {
8              data: [0; DEBUG_BUF_SIZE],
9              len: 0,
10         }
11     }
12     pub fn clear(&mut self) {
13         self.len = 0;
14     }
15     pub fn write_str(&mut self, s: &str) {
16         ...
17     }
18     pub fn as_str(&self) -> &str {
```

```
19          core::str::from_utf8(&self.data[..self.len]).unwrap_or("[Invalid
                UTF-8]")
20      }
21  }
```

En el código incluido anteriormente se muestra la estructura del buffer de depurado, consistiendo este de el tamaño ocupado del buffer junto con un vector de caracteres con el tamaño del buffer de depurado.

Posteriormente se encuentran una serie de funciones cuyo código no tiene importancia, puesto que la implementación es trivial por ser similar a varias funciones que hemos implementado anteriormente.

debug_log

```
1  pub fn debug_log(s: &str) {
2      {
3          let mut buf = DEBUG_BUFFER.lock();
4          buf.clear();
5          buf.write_str(s);
6      }
7
8      trigger_debug();
9
10     restore_screen_buffer();
11 }
```

La función `debug log` es la encargada de iniciar la secuencia de acciones, cuando se quiera lanzar una depuración se llamará desde el componente que sea, una llamada a esta función pasándole como argumento la cadena que se quiera imprimir, o en otras palabras, el mensaje de depurado.

Esta función guarda sobre el buffer el mensaje y llama a la función trigger debug.

trigger_debug

```
1  pub fn trigger_debug() {
2      let mut saved = SAVED_SCREEN.lock();
3
4      unsafe {
5          core::ptr::copy_nonoverlapping(
6              VGA_BUFFER_ADDR as *const u8,
7              saved.as_mut_ptr(),
8              BUFFER_SIZE,
9          );
10     }
11
12     interrupts::unmask_irq0();
13     DEBUG_MODE.store(true, SeqCst);
14     draw_debug_buffer();
15
```

```
    loop {
        if !DEBUG_MODE.load(SeqCst) {
            break;
        }
        hlt();
    }
}
```

Esta función se encarga de activar el modo de depuración, un booleano global que controlará el funcionamiento de componentes como la interrupción de teclado, permitiendo únicamente procesar la pulsación de la tecla q (quit) durante el tiempo que este modo esté activo.

Posteriormente se desactiva la interrupción de reloj, que haría que este bucle se ejecute innecesariamente ya que la única acción que puede liberar el bucle es la pulsación de la tecla q.

draw_debug_buffer

```
pub fn draw_debug_buffer() {
    let buf = DEBUG_BUFFER.lock();
    for _ in 0..50 {
        println!("");
    }
    print!("{}", buf.as_str());
}
```

Esta función limpia la pantalla e imprime el mensaje que hay almacenado en el buffer, es la función que se usa para mostrar por terminal los mensajes de depuración.

restore_screen_buffer

```
pub fn restore_screen_buffer() {
    let saved = SAVED_SCREEN.lock();

    unsafe {
        core::ptr::copy_nonoverlapping(
            saved.as_ptr(),
            VGA_BUFFER_ADDR as *mut u8,
            BUFFER_SIZE,
        );
    }

    let mut writer = WRITER.lock();
    writer.cursor_position(2);

    interrupts::mask_irq0();
}
```

Esta función se encarga de reactivar las interrupciones de reloj, además

Figura 6.1: Estado inicial, la terminal espera la introducción de un comando

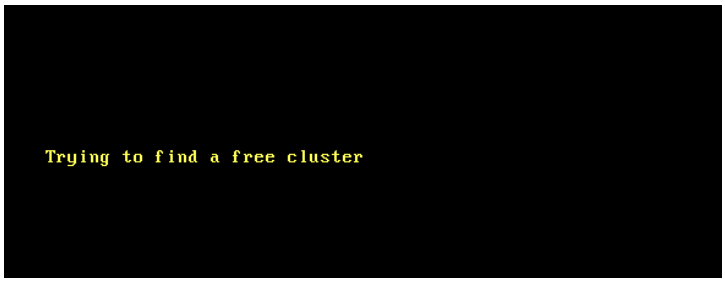

Figura 6.2: Primer mensaje de depuración, el sistema de archivos está intentando encontrar un *cluster* libre donde alojar el directorio.

también copia a la memoria de vídeo el valor de la memoria guardada en el buffer auxiliar y posiciona el cursor en la posición que debería tomar al volver de el modo de depuración.

6.3.10. Ejemplo

A continuación se encuentra un ejemplo de esta ejecución, en el que se crea un directorio llamado A que pertenece al directorio raíz.

6.3.11. Sistema de Archivos

Este archivo contiene toda la implementación del sistema de archivos FAT32, en él encontramos principalmente 5 partes importantes

Estructuras

Las estructuras básicas están definidas siguiendo la descripción del sistema FAT32 de la página web [18]

DirEntry

```
pub struct DirEntry {
    pub name: [u8; 11],
```

Figura 6.3: Segundo mensaje de depuración, el sistema de archivos ha encontrado que el primer *cluster* disponible es el *cluster* con id 6

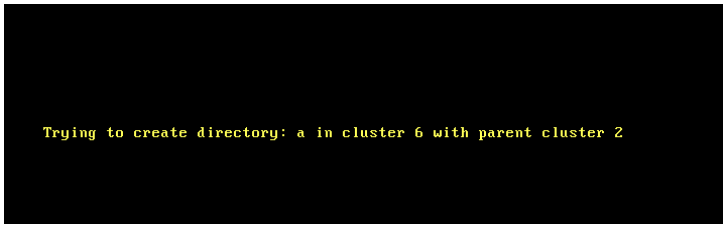

Figura 6.4: Último mensaje de depuración, se informa del nombre del directorio, el clúster donde se va a alojar y el clúster inicial del directorio al que pertenece.

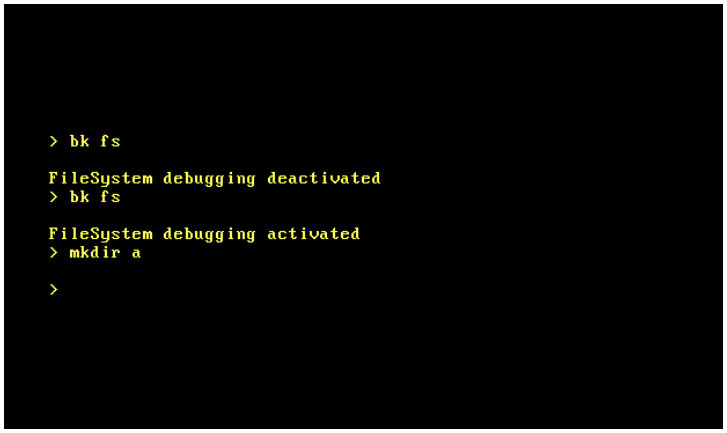

Figura 6.5: La terminal tras la ejecución, esperando el siguiente comando.

```
3     pub attr: u8,
4     pub reserved: u8,
5     pub creation_time_tenths: u8,
6     pub creation_time: u16,
7     pub creation_date: u16,
8     pub last_access_date: u16,
9     pub first_cluster_high: u16,
10    pub write_time: u16,
11    pub write_date: u16,
12    pub first_cluster_low: u16,
13    pub file_size: u32,
14 }
```

BiosParameterBlock

```
1 #[derive(Copy, Clone, Debug)]
2 #[repr(C)]
3 pub struct BiosParameterBlock {
4     pub _jmp: [u8; 3],
5     pub _oem: [u8; 8],
6     pub bytes_per_sector: u16,
7     pub sectors_per_cluster: u8,
8     pub reserved_sectors: u16,
9     pub fat_table_count: u8,
10    pub root_entries: u16,
11    pub total_sectors_16: u16,
12    pub media_descriptor: u8,
13    pub fat_size_16: u16,
14    pub sectors_per_track: u16,
15    pub heads_on_media: u16,
16    pub hidden_sectors: u32,
17    pub total_sectors_32: u32,
18 }
```

ExtenderBootRecord32

```
1 #[derive(Copy, Clone, Debug)]
2 #[repr(C)]
3 pub struct ExtendedBootRecord32 {
4     pub fat_size_32: u32,
5     pub ext_flags: u16,
6     pub fat_version: u16,
7     pub root_cluster: u32,
8     pub fs_info: u16,
9     pub backup_boot: u16,
10    pub _reserved: [u8; 12],
11    pub drive_number: u8,
12    pub _win_nt_flags: u8,
13    pub signature: u8,
14    pub volume_id: u32,
15    pub volume_label: [u8; 11],
16    pub fs_id: [u8; 8],
17 }
```

Trait BlockDevice

```
pub trait BlockDevice {
    fn read_sector(&mut self, lba: u64, buf: &mut [u8;512]) -> Result<(),
        ()>;
    fn write_sector(&mut self, lba: u64, buf: &[u8;512]) -> Result<(), ()
        >;
    fn raw_data_mut(&mut self) -> &mut [u8];
}
```

Como se ha explicado anteriormente, un *trait* es similar a una interfaz en el lenguaje de programación C, al definir un *trait* es posible por ejemplo definir estructuras que dependan de dispositivos que tengan este *trait* sin influir cómo el mismo haya sido implementado siempre que sea de manera correcta.

Esto significa que por ejemplo para definir un sistema de archivos no sería necesario definirlo de nuevo si cambia el tipo de disco, únicamente será necesario implementar el *trait* para cada disco que queramos utilizar.

FileSystem

```
pub struct FileSystem<'a, D: BlockDevice> {
    device: &'a mut D,
    pub bpb: BiosParameterBlock,
    pub ebr: ExtendedBootRecord32,
    pub fat_start: u32,
    cluster_heap_start: u32,
    root_dir_cluster: u32,
}
```

Precisamente como se ha explicado en el apartado anterior, para definir el sistema de archivos no vamos a definir el tipo del dispositivo sobre el que se monta, únicamente es necesario definir las dos estructuras mencionadas anteriormente, que son básicas para cualquier sistema FAT32 junto con algunos datos más.

El dispositivo (device en este caso) no tiene tipo, sino que podemos utilizar cualquier tipo que implemente el trait `BlockDevice`.

RamDisk

```
pub struct RamDisk {
    data: Vec<u8>,
}
```

Como se había descrito en capítulos anteriores, este dispositivo no va a ser más que un *array* de valores del tipo `u8` (enteros sin signo de 8 bits) sobre el heap, por lo tanto no es un dispositivo real, sin embargo para la implementación del sistema de archivos es más que suficiente, ya que desde

el punto de vista de el mismo, este disco es tan real como un disco duro físico.

Implementación del sistema de archivos

El sistema de archivos implementa una serie de módulos con funcionalidades muy similares, por ejemplo `create_file` y `create_dir`, que son esencialmente la misma función únicamente cambiando el tipo de archivo que se almacena.

Por lo tanto únicamente se explicarán en detalle las funciones principales o que implementen lógica nueva.

init_fats

```
pub fn init_fats(&mut self) {
    let bytes_per_sector = self.bpb.bytes_per_sector as usize;

    let fat_size_sectors = match self.bpb.fat_size_16 {
        0 => self.ebr.fat_size_32 as usize,
        n => n as usize,
    };

    let media_descriptor = self.bpb.media_descriptor;

    let reserved = self.bpb.reserved_sectors as usize;
    let fat_size_bytes = fat_size_sectors * bytes_per_sector;

    let data = self.device.raw_data_mut();

    let mut byte_offset = reserved * bytes_per_sector;

    for i in 0..self.bpb.fat_table_count as usize {
        let end = byte_offset + fat_size_bytes;
        assert!(
            end <= data.len(),
            "FAT copy {} out of disk bounds (offset {} len {})",
            i,
            byte_offset,
            fat_size_bytes
        );

        let fat_slice = &mut data[byte_offset..end];
        Self::init_fat_helper(fat_slice, media_descriptor);

        byte_offset += fat_size_bytes;
    }
}
```

El código anterior inicializa todas las tablas de archivos (sean 2 o más) . Primero se calcula dónde empieza cada una de las tablas y posteriormente se ejecuta una función auxiliar que se encargará de hacer la inicialización obteniendo como argumentos las posiciones de memoria que hay que inicializar y el descriptor de medios.

write_fat_entry

```
fn write_fat_entry(&mut self, cluster_idx: u32, value: u32) {
    let reserved = self.bpb.reserved_sectors as usize;
    let bytes_per_sector = self.bpb.bytes_per_sector as usize;

    let fat_table_count = self.bpb.fat_table_count;
    let fat_size_sectors = match self.bpb.fat_size_16 {
        0 => self.ebr.fat_size_32 as usize,
        n => n as usize,
    };

    let fat_bytes_len = fat_size_sectors * bytes_per_sector;

    let data: &mut [u8] = self.device.raw_data_mut();
    let first_fat_offset = reserved * bytes_per_sector;

    let bytes = value.to_le_bytes();

    for i in 0..fat_table_count {
        let fat_i_offset = first_fat_offset + (i as usize * fat_bytes_len)
            ;
        let entry_offset = fat_i_offset + (cluster_idx as usize * 4);
        data[entry_offset..entry_offset + 4].copy_from_slice(&bytes);
    }
}
```

La función de más arriba se encarga de escribir entradas sobre las tablas de archivos. Un archivo en una tabla está representado por el valor del primer *cluster* de la cadena, esta función se encarga de escribir ese valor sobre los sectores del disco.

free_cluster_chain

```
pub fn free_cluster_chain(&mut self, start_cluster: u32) -> Result<(), ()>
     {
    let reserved = self.bpb.reserved_sectors as usize;
    let bytes_per_sector = self.bpb.bytes_per_sector as usize;

    let fat_size_sectors = match self.bpb.fat_size_16 {
        0 => self.ebr.fat_size_32 as usize,
        n => n as usize,
    };

    let first_fat_offset = reserved * bytes_per_sector;

    let fat_bytes_len = fat_size_sectors * bytes_per_sector;
    let fat_table_count = self.bpb.fat_table_count;
    let data = self.device.raw_data_mut();


    let mut current_cluster = start_cluster;

    while current_cluster < 0x0FFF_FFF8 {
        let next_cluster = {
            let fat_offset = first_fat_offset + (current_cluster as usize
                * 4);
            let entry = u32::from_le_bytes(
```

```
                data[fat_offset..fat_offset+4].try_into().unwrap()
            );
            entry
        };

        for i in 0..fat_table_count {
            let fat_offset = first_fat_offset + (i as usize *
                fat_bytes_len);
            let entry_offset = fat_offset + (current_cluster as usize * 4)
                ;
            data[entry_offset..entry_offset+4].copy_from_slice(&0u32.
                to_le_bytes());
        }

        current_cluster = next_cluster;
    }
    Ok(())
}
```

Dado que los archivos se representan como cadenas de *clusters*, es necesario liberar todos los elementos de esta cadena cuando se quiera liberar un archivo. Esta función se encarga de recorrer esa cadena, guardar el siguiente *cluster* y eliminar el *cluster* actual siempre que no sea el código de final de *cluster*.

return_tree

```
pub fn return_tree(&mut self, cluster_idx: u32, depth: usize) -> String{
    let entries = self.read_dir_entries(cluster_idx as usize);

    let mut out = String::new();

    for entry in entries {
        for _ in 0..depth {
            out.push_str("\\ ");
        }

        if entry.is_directory() {
            let _ = writeln!(out, "* {}", entry.get_name().unwrap().
                trim_end());
            let name = entry.get_name().unwrap();

            if name != "." && name != ".." {
                let subtree = self.return_tree(entry.first_cluster(),
                    depth + 1);
                out.push_str(&subtree);
            }

        } else {
            let _ = writeln!(out, "* {}", entry.get_name().unwrap());
        }
    }
    out
}
```

Similar al comando *tree* en Linux, esta función permite obtener como un

string la estructura de los archivos. De manera recursiva se accede a los directorios que se van encontrando al recorrer las entradas del root. Posteriormente se devuelve la estructura de los archivos y directorios que se encuentran recursivamente.

create_dir

```
pub fn create_dir(&mut self, parent_dir_cluster: u32, filename: &str) -> Result<(), ()> {
    let cluster = self.allocate_cluster().unwrap();

    let entry: DirEntry = DirEntry::new(str_to_fat_name(filename), cluster, 0x10);

    self.zero_cluster_data(cluster as usize);


    let dot = DirEntry::new(*b".          ", cluster as u32, 0x10);
    let dot2 = DirEntry::new(*b"..         ", parent_dir_cluster as u32, 0x10);

    self.allocate_dir_entry(dot, cluster as usize).unwrap();
    self.allocate_dir_entry(dot2, cluster as usize).unwrap();

    self.allocate_dir_entry(entry, parent_dir_cluster as usize);
    Ok(())
}
```

Un directorio y un archivo en FAT32 usan la misma estructura, lo único que diferencia estos dos tipos de entradas es el valor del byte `attr` en la entrada, para archivos toma el valor `0x20` y para archivos el valor `0x10`.

Esta función se encarga de alojar el *cluster*, crear la estructura que será almacenada en el directorio padre y alojar en este nuevo directorio una referencia al directorio padre `(..)` y a si mismo `(.)`.

create_root_dir

```
pub fn create_root_dir(&mut self) -> Result<(), ()> {
    let root_cluster = self.ebr.root_cluster as usize;

    self.zero_cluster_data(root_cluster);

    let dot = DirEntry::new(*b".          ", root_cluster as u32, 0x10);
    let dot2 = DirEntry::new(*b"..         ", root_cluster as u32, 0x10);

    self.allocate_dir_entry(dot, root_cluster).unwrap();
    self.allocate_dir_entry(dot2, root_cluster).unwrap();


    self.write_fat_entry(root_cluster as u32, 0x0FFF_FFFFu32);

    Ok(())
}
```

Esta función es similar a la anterior con la diferencia de que todas las referencias apuntan al *root*, y no es necesario almacenar la entrada en ningún directorio ya que el directorio raíz no pertenece a ningún directorio de orden superior.

find_dir_in

```
pub fn find_dir_in(&mut self, cluster: u32, name: &str) -> Option<u32> {
    let entries = self.read_dir_entries(cluster as usize);
    for entry in entries {
        if entry.is_directory() {
            if let Ok(entry_name) = entry.get_name() {
                if entry_name.trim_end() == name {
                    return Some(entry.first_cluster());
                }
            }
        }
    }
    None
}
```

Esta función es necesaria para la implementación del comando `cd` (*change directory*), cuando se quiere buscar un directorio es más conveniente buscarlo utilizando su nombre que su clúster. Esta función recorre los elementos de un directorio y compara el nombre de estos con la cadena que se ha enviado como argumento.

read_dir_entries

```
pub fn read_dir_entries(&mut self, cluster_idx: usize) -> Vec<DirEntry> {
    let reserved = self.bpb.reserved_sectors as usize;
    let bytes_per_sector = self.bpb.bytes_per_sector as usize;
    let sectors_per_cluster = self.bpb.sectors_per_cluster as usize;
    let fat_table_count = self.bpb.fat_table_count as usize;

    let fat_size_sectors = if self.bpb.fat_size_16 == 0 {
        self.ebr.fat_size_32 as usize
    } else {
        self.bpb.fat_size_16 as usize
    };

    let data_start = (reserved + fat_table_count * fat_size_sectors) *
        bytes_per_sector;

    let cluster_offset = (cluster_idx - 2) * sectors_per_cluster *
        bytes_per_sector;

    let start_cluster = data_start + cluster_offset;
    let cluster_size = sectors_per_cluster * bytes_per_sector;

    let data = self.device.raw_data_mut();

    let entries_per_cluster = cluster_size / 32;

    let mut entries = Vec::new();

```

```
26     for entry_idx in 0..entries_per_cluster {
27         let byte_offset = start_cluster + (entry_idx * 32);
28 
29         let dir_slice = &data[byte_offset..byte_offset + 32];
30 
31         let first_byte = dir_slice[0];
32 
33         if first_byte == 0x00 {
34             break; // no more entries
35         }
36         if first_byte == 0xE5 {
37             continue; // deleted entry, skip
38         }
39 
40         let entry = DirEntry::deserialize(dir_slice.try_into().unwrap());
41         entries.push(entry);
42     }
43     entries
44 }
```

esta función auxiliar devuelve un vector de entradas pertenecientes a un directorio, es una función auxiliar crucial para la implementación de `cd`, `ls` y cualquier comando que pretenda interactuar con las entradas de un directorio.

new

```
1  pub fn new(device: &'a mut D) -> Result<Self, ()> {
2      let mut sector = [0u8; 512];
3      device.read_sector(0, &mut sector)?;
4 
5      let bpb = BiosParameterBlock::from_bytes(&sector[0..36])?;
6 
7      let ebr = ExtendedBootRecord32::from_bytes(&sector[36..90])?;
8 
9 
10     let fat_start = bpb.reserved_sectors as u32;
11     let cluster_heap_start = fat_start + (ebr.fat_size_32 * bpb.
           fat_table_count as u32);
12     let root_dir_cluster = ebr.root_cluster;
13 
14     Ok(Self {
15         device,
16         bpb,
17         ebr,
18         fat_start,
19         cluster_heap_start,
20         root_dir_cluster,
21     })
22 }
```

La función de inicialización lee los valores de los bloques de arranque del sistema de Archivos para poder trabajar correctamente sobre él. Para ello se extrae de la memoria los bytes en los que estos están contenidos y se devuelve un sistema de archivos con todos los valores asignados.

Inicialización del disco

```
impl RamDisk {
    #[allow(clippy::too_many_arguments)]
    pub fn new(
        size_in_sectors: usize,
        sectors_per_cluster: u8,
        reserved_sectors: u16,
        fat_count: u8,
        fat_size: u32,
        root_cluster: u32,
        fs_info_sector: u16,
        backup_boot_sector: u16,
        volume_id: u32,
        volume_label: [u8; 11],
    ) -> Self {

        let total_sectors = reserved_sectors as usize + (fat_count as usize * fat_size as usize);
        if total_sectors > size_in_sectors {
            panic!(
                "Provided size_in_sectors ({}) is too small; at least {} sectors are required to fit reserved + FAT + data regions.",
                size_in_sectors, total_sectors
            );
        }
        let mut data = vec![0u8; size_in_sectors * 512];

        println!("{}", size_in_sectors * 512);

        // Basic jump and OEM
        data[0] = 0xEB;
        data[1] = 0x58;
        data[2] = 0x90;
        data[3..11].copy_from_slice(b"MSWIN4.1");

        // BIOS Parameter Block
        data[11..13].copy_from_slice(&512u16.to_le_bytes()); // bytes/sector
        data[13] = sectors_per_cluster;
        data[14..16].copy_from_slice(&reserved_sectors.to_le_bytes());
        data[16] = fat_count;
        data[17..19].copy_from_slice(&0u16.to_le_bytes()); // root entries

        data[19..21].copy_from_slice(&(if size_in_sectors < 65536 {
            size_in_sectors as u16} else {0}).to_le_bytes());

        data[21] = 0xF8; // media descriptor
        data[22..24].copy_from_slice(&0u16.to_le_bytes()); // fat_size_16
        data[24..26].copy_from_slice(&63u16.to_le_bytes()); // sectors/track
        data[26..28].copy_from_slice(&255u16.to_le_bytes()); // heads
        data[28..32].copy_from_slice(&0u32.to_le_bytes()); // hidden sectors
        data[32..36].copy_from_slice(&(if size_in_sectors >= 65536 {
            size_in_sectors as u32
        } else {
                0
            })
            .to_le_bytes());

```

```
55          // Extended Boot Record
56          data[36..40].copy_from_slice(&fat_size.to_le_bytes());
57          data[40..42].copy_from_slice(&0u16.to_le_bytes()); // ext flags
58          data[42..44].copy_from_slice(&0u16.to_le_bytes()); // fat version
59          data[44..48].copy_from_slice(&root_cluster.to_le_bytes());
60          data[48..50].copy_from_slice(&fs_info_sector.to_le_bytes());
61          data[50..52].copy_from_slice(&backup_boot_sector.to_le_bytes());
62
63          // Drive/boot fields
64          data[64] = 0x80;
65          data[65] = 0x00; // Windows NT flags
66          data[66] = 0x29; // signature
67          data[67..71].copy_from_slice(&volume_id.to_le_bytes());
68          data[71..82].copy_from_slice(&volume_label);
69          data[82..90].copy_from_slice(b"FAT32   ");
70
71          // Boot sector signature
72          data[510] = 0x55;
73          data[511] = 0xAA;
74
75          Self { data }
76      }
77  }
```

En la función superior se encuentra la inicialización del disco duro que se ha implementado sobre el heap, para ello se comprueba si el total de sectores a reservar es el mínimo necesario, en caso contrario se detiene la ejecución.

Si fuera suficiente se procede a reservar la memoria necesaria para almacenar el número de sectores que se han especificado. Posteriormente se inicializa todo el disco con los valores.

En un disco real, no volátil este sería el proceso de formateo del disco, en el que se escriben los valores necesarios sobre él, y posteriormente únicamente haría falta leerlo. Sin embargo como este disco está simulado sobre el heap será necesario inicializarlo cada vez que se quiera ejecutar el sistema operativo.

Cada uno de los parámetros está especificado ya sea con un comentario o en el propio nombre de los argumentos, el formato de los bytes sigue el formato especificado por la comunidad de desarrolladores de sistemas operativos[18].

Implementación del trait BlockDevice para RamDisk

```
1  impl BlockDevice for RamDisk {
2      fn read_sector(&mut self, lba: u64, buf: &mut [u8; 512]) -> Result<(),
           ()> {
3          let start = (lba as usize) * 512;
4          let end = start + 512;
5          if end > self.data.len() {
6              return Err(());
7          }
8          buf.copy_from_slice(&self.data[start..end]);
9          Ok(())
```

```
10      }
11
12      fn write_sector(&mut self, lba: u64, buf: &[u8; 512]) -> Result<(), ()
            > {
13          let start = (lba as usize) * 512;
14          let end = start + 512;
15          if end > self.data.len() {
16              return Err(());
17          }
18          self.data[start..end].copy_from_slice(buf);
19          Ok(())
20      }
21
22      fn raw_data_mut(&mut self) -> &mut [u8] {
23          &mut self.data[..]
24      }
25  }
```

Como se explicó anteriormente el objetivo de utilizar los *traits* en Rust es la posibilidad de utilizar diferentes infraestructuras (como por ejemplo el disco sobre la RAM o un Disco Duro físico) siempre y cuando se implemente el *trait* en específico.

En este ejemplo vemos la implementación del *trait* `BlockDevice` sobre `RamDisk`, de esta manera se permite que `FileSystem` utilice una instancia de la estructura `RamDisk` como su elemento de nombre *device*.

Si se quisiera utilizar un disco duro únicamente sería necesario implementar estas tres funciones con ese *trait* para el nuevo tipo.

La implementación de estas funciones es simple, únicamente se busca devolver los datos almacenados en una parte de los datos del dispositivos y escribir sobre la misma, por lo tanto la implementación es trivial, consiste en leer del vector o escribir sobre el vector.

Implementación del trait Display para varias estructuras

El *trait* Display se usa en *fmt* para permitir usar nuestras variables como argumento de entrada para nuestros *prints*, por lo tanto es necesario definir este *trait* para facilitar la depuración del código, ya que se puede acceder a los valores de las funciones.

A continuación se muestra el código necesario para poder utilizar estas estructuras como argumentos para un formateado.

DirEntry

```
1  impl fmt::Display for DirEntry {
2      fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
3          let cluster = ((self.first_cluster_high as u32) << 16) | (self.
               first_cluster_low as u32);
4
```

```
        write!(
            f,"DirEntry {{
                Name: \"{}\", FC: {}
                }}", self.get_name().unwrap(), cluster
        )
    }
}
```

BiosParameterBlock

```
impl fmt::Display for BiosParameterBlock {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        let oem_str = core::str::from_utf8(&self._oem)
            .unwrap_or("<invalid utf8>");

        write!(
            f,
            "BiosParameterBlock {{
JMP: {:02X?}
OEM: {}
Bytes per sector: {}
Sectors per cluster: {}
Reserved sectors: {}
FAT table count: {}
Root entries: {}
Total sectors 16: {}
Media descriptor: 0x{:02X}
FAT size 16: {}
Sectors per track: {}
Heads on media: {}
Hidden sectors: {}
Total sectors 32: {}
}}",
            &self._jmp,
            oem_str,
            self.bytes_per_sector,
            self.sectors_per_cluster,
            self.reserved_sectors,
            self.fat_table_count,
            self.root_entries,
            self.total_sectors_16,
            self.media_descriptor,
            self.fat_size_16,
            self.sectors_per_track,
            self.heads_on_media,
            self.hidden_sectors,
            self.total_sectors_32,
        )
    }
}
```

ExtendedBootRecord32

```
impl fmt::Display for ExtendedBootRecord32 {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        let volume_label_str = core::str::from_utf8(&self.volume_label)
            .unwrap_or("<invalid utf8>");
        let fs_id_str = core::str::from_utf8(&self.fs_id)
```

```
6            .unwrap_or("<invalid utf8>");
7
8        write!(
9            f,
10           "ExtendedBootRecord32 {{
11 FAT size 32: {}
12 Ext flags: {}
13 FAT version: {}
14 Root cluster: {}
15 FS info sector: {}
16 Backup boot sector: {}
17 Drive number: {}
18 Signature: 0x{:02X}
19 Volume ID: 0x{:08X}
20 Volume Label: {}
21 FS ID: {}
22 }}",
23           self.fat_size_32,
24           self.ext_flags,
25           self.fat_version,
26           self.root_cluster,
27           self.fs_info,
28           self.backup_boot,
29           self.drive_number,
30           self.signature,
31           self.volume_id,
32           volume_label_str,
33           fs_id_str,
34       )
35   }
36 }
```

Capítulo 7

Pruebas

A lo largo de la implementación del sistema se han diseñado una serie de pruebas que prueban cómo los componentes interactúan entre ellos y cómo los componentes en sí están implementados. Para ello se han diseñado una serie de tests de integración y unitarios, que servirán a modo de comprobación para asegurar que tras implementar una actualización el sistema sigue funcionando con normalidad.

7.1. Tests de integración

Como se explicó anteriormente, en el módulo lib se definió una manera personalizada de tratar los tests, lo que se hará será recorrer los archivos buscando el atributo `#[test_case]`, adicionalmente en los archivos del directorio tests se ejecutará el main o _start. Es precisamente esta ejecución del main lo que nos permitirá crear las condiciones deseadas para poner a prueba los tests.

7.1.1. heap_allocation

Entry poin

```
fn main(boot_info: &'static BootInfo) -> ! {
    use ups::allocator;
    use ups::memory::{self, BootInfoFrameAllocator};
    use x86_64::VirtAddr;

    ups::init();
    let phys_mem_offset = VirtAddr::new(boot_info.physical_memory_offset);
    let mut mapper = unsafe { memory::init(phys_mem_offset) };
    let mut frame_allocator = unsafe {
        BootInfoFrameAllocator::init(&boot_info.memory_map)
    };
    allocator::init_heap(&mut mapper, &mut frame_allocator)
```

```
13          .expect("heap initialization failed");
14
15      test_main();
16      loop {}
17  }
```

Como podemos observar en la función anterior, estamos inicializando la memoria virtual para poder hacer pruebas sobre la paginación, al hacer esto previo a ejecutar los tests podemos inicializar los vectores, que permitirá utilizar paginación y el heap en nuestros tests.

```
1  #[test_case]
2  fn simple_allocation() {
3      let heap_value_1 = Box::new(41);
4      let heap_value_2 = Box::new(13);
5      assert_eq!(*heap_value_1, 41);
6      assert_eq!(*heap_value_2, 13);
7  }
8
9  #[test_case]
10 fn large_vec() {
11     let n = 1000;
12     let mut vec = Vec::new();
13     for i in 0..n {
14         vec.push(i);
15     }
16     assert_eq!(vec.iter().sum::<u64>(), (n - 1) * n / 2);
17 }
18
19
20 #[test_case]
21 fn many_boxes() {
22     for i in 0..100000 {
23         let x = Box::new(i);
24         assert_eq!(*x, i);
25     }
26 }
```

Las tres funciones anteriores hacen uso del heap, por lo que vemos que es importante que antes de ejecutarlas inicialicemos la memoria y la paginación.

En el primer test creamos dos valores en el heap y comprobamos que se hayan escrito correctamente, el segundo aloja valores dentro de un vector y posteriormente comprueba que la suma sea correcta. Finalmente el último test aloja un gran cantidad de valores y comprueba que se hayan alojado de manera correcta todos ellos en la memoria.

7.1.2. stack_overflow

```
1  extern "x86-interrupt" fn test_double_fault_handler(
2      _stack_frame: InterruptStackFrame,
3      _error_code: u64,
4  ) -> ! {
```

```
5     serial_println!("{}", Green("[ok]"));
6     exit_qemu(QemuExitCode::Success);
7     loop {}
8 }
9
10 #[unsafe(no_mangle)]
11 pub extern "C" fn _start() -> ! {
12     serial_print!("stack_overflow::stack_overflow...\t");
13
14     ups::gdt::init();
15     init_test_idt();
16
17     stack_overflow();
18
19     panic!("Execution continued after stack overflow");
20 }
21
22 #[allow(unconditional_recursion)]
23 fn stack_overflow() {
24     stack_overflow();
25     volatile::Volatile::new(0).read();
26 }
```

En este archivo de testado no existe un test case, ya que el main es el que ejecutará la prueba, primero inicializamos las interrupciones y después forzamos un desbordamiento de pila, para poder recibir correctamente este error lo que haremos será definir la excepción doble para que imprima por el puerto serie que todo ha salido como debería y finalice la ejecución de qemu.

Esto se hace debido a que cuandose intente alojar en la pila varias veces sin tratar el desbordamiento recibiremos una excepción doble, que es lo que debería ocurrir, así que imprimimos que el flujo del programa ha sido satisfactorio.

7.2. Tests unitarios

En el caso de que el archivo no defina una función main o start se utilizará la función que se haya definido por defecto, en nuestro caso será `test_kernel_main`. Tras la ejecución de esta función se ejecutarán una a una nuestras funciones con el atributo `#[test_case]`, por ejemplo:

7.2.1. vga_buffer

```
1 #[test_case]
2 fn test_println_output() {
3     use core::fmt::Write;
4     use x86_64::instructions::interrupts;
5
6     let s = "Some test string that fits on a single line";
7     interrupts::without_interrupts(|| {
8         let mut writer = WRITER.lock();
```

```
9           writeln!(writer, "\n{}", s).expect("writeln failed");
10          for (i, c) in s.chars().enumerate() {
11              let screen_char = writer.buffer.chars[BUFFER_HEIGHT - 2][i].
                    read();
12              assert_eq!(char::from(screen_char.ascii_character), c);
13          }
14      });
15  }
```

En este ejemplo únicamente se hace uso de las variables que se han definido en esta librería, de esta manera esto define un test unitario, que únicamente interactúa con los componentes necesarios para probar esta función.

Capítulo 8

Conclusiones y trabajos a futuro

En este capítulo se hará una recolección de lo que se ha logrado mediante la implementación de este proyecto, las conclusiones que se han sacado y qué trabajo podría desarrollarse en un futuro para hacer que esta herramienta se vuelva aún más potente y sencilla de usar.

8.1. Objetivos alcanzados

Se ha logrado desarrollar un sistema operativo, que simula los componentes típicos de un sistema real y que es capaz de alojar archivos, información y de interactuar mediante un teclado.

Se ha conseguido desarrollar una interfaz de depurado simple, que muestra por pantalla información sobre los componentes y que permite elegir qué componentes queremos detener en cada momento.

Se ha podido proveer de una serie de programas básicos que hagan uso de estos componentes para poder comprobar que el funcionamiento de los mismos es correcto, además de devolver información relevante sobre la ejecución del código.

8.2. Trabajos a futuro

Hay varias ideas que podrían resultar interesantes para seguir desarrollando este proyecto, a continuación se listan algunas de ellas:

- **Planificador y programas**

En este sistema no se ha implementado la lógica que permite ejecutar programas almacenados en memoria, tampoco controlar qué programa tiene el derecho en cada momento de ejecutarse sobre la CPU. Sería interesante implementar la lógica para hacer funcionar los mismos, de manera que sea posible ejecutar programas propios sobre el sistema.

- **Implementación de controlador de *framebuffer***

  Para el sistema que se ha diseñado ha sido más que suficiente la capacidad que brinda el uso de la memoria VGA para imprimir caracteres en la pantalla, aunque quizás si se quisieran implementar programas más avanzados como por ejemplo una simulación de video, esta memoria no sería suficiente.

- **Soporte multihebra**

  Este sistema tampoco es capaz, debido principalmente a la falta de una estructura para procesos y planificación, ejecutar varias hebras de procesos, esto es muy interesante para ejecutar programas modernos y ver cómo funcionan exactamente, por ello sería también una buena idea de implementación en un futuro.

8.3. Conclusión

La implementación de este sistema operativo ha sido un reto de investigación tanto como de codificación, que ha permitido adquirir unos conocimientos y habilidades muy valorables. En lo personal, el desarrollo de este proyecto no sólo me ha permitido aprender a investigar, documentar y desarrollar un proyecto por mi cuenta, sino que me ha permitido profundizar en temas que se han tratado durante el grado de manera tangencial.

Por ejemplo he podido comprender cómo se organizan los archivos en un sistema operativo a nivel de hardware, también he comprendido cómo funciona el heap y qué componentes del sistema son necesarios para implementarlo, y también cómo la paginación es vital para un sistema operativo moderno y qué sistemas permiten que sea competitiva en comparación con el resto de organizaciones de memoria virtual.

Adicionalmente se ha logrado desarrollar una herramienta útil tanto para nuevos estudiantes como para expertos interesados en reforzar la base de sus conocimientos. La creación de un producto como lo es este sistema operativo permitirá a los estudiantes interesados en el desarrollo de sistemas operativos aprender de una manera más práctica e interactiva en el futuro.

Bibliografía

[1] André Machado. Minix: History, architecture, and comparisons with gnu/linux, 2024. Accessed: 11 de junio de 2025. URL: https://machaddr.substack.com/p/minix-history-architecture-and-comparisons.

[2] Os Dev Wiki. Microkernel, 2021. Accessed: 11 de junio de 2025. URL: https://osdev.wiki/wiki/Microkernel.

[3] Os Dev Wiki. Xv6, 2024. Accessed: 11 de junio de 2025. URL: https://osdev.wiki/wiki/Xv6.

[4] Os Dev Wiki. Monolithic kernel, 2021. Accessed: 11 de junio de 2025. URL: https://osdev.wiki/wiki/Monolithic_Kernel.

[5] Robert Love. *Linux Kernel Development*. Addison-Wesley, 3 edition, 2010.

[6] Os Dev Wiki. Printing to screen, 2024. Accessed: 6 de junio de 2025. URL: https://osdev.wiki/wiki/Printing_To_Screen.

[7] Os Dev Wiki. Drawing in a linear framebuffer, 2025. Accessed: 6 de junio de 2025. URL: https://osdev.wiki/wiki/Drawing_In_a_Linear_Framebuffer#Locating_Video_Memory.

[8] Os Dev Wiki. Interrupts, 2024. Accessed: 11 de junio de 2025. URL: https://osdev.wiki/wiki/Interrupts.

[9] Daniel P. Bovet and Marco Cesati. *Understanding the Linux Kernel*. O'Reilly Media, 3 edition, 2005.

[10] Os Dev Wiki. Exceptions, 2024. Accessed: 11 de junio de 2025. URL: https://osdev.wiki/wiki/Exceptions.

[11] Os Dev Wiki. Interrupts, 2024. Accessed: 11 de junio de 2025. URL: https://osdev.wiki/wiki/Exceptions#Double_Fault.

[12] Os Dev Wiki. Shared_memory, 2023. Accessed: 11 de junio de 2025.

URL: https://osdev.wiki/wiki/Shared_Memory.

[13] Os Dev Wiki. Segmentation, 2025. Accessed: 11 de junio de 2025. URL: https://osdev.wiki/wiki/Segmentation.

[14] Os Dev Wiki. Paging, 2025. Accessed: 11 de junio de 2025. URL: https://osdev.wiki/wiki/Paging.

[15] Philipp Oppermann. Introduction to paging, 2019. Accessed: 7 de junio de 2025. URL: https://os.phil-opp.com/paging-introduction/#multilevel-page-tables.

[16] Abraham Silberschatz, Peter B. Galvin, and Greg Gagne. *Operating System Concepts*. John Wiley & Sons, 10 edition, 2022.

[17] Breakthrough (User with the most rated answer). How much time until an unused hard drive loses its data?, 2011. Accessed: 8 de junio de 2025. URL: https://superuser.com/questions/284427/how-much-time-until-an-unused-hard-drive-loses-its-data.

[18] Os Dev Wiki. Fat, 2025. Accessed: 8 de junio de 2025. URL: https://wiki.osdev.org/FAT#Standard_8.3_format.

[19] Kevin Purdy. As the kernel turns: Rust in linux saga reaches the "linus in all-caps" phase, 2025. Accessed: 8 de junio de 2025. URL: https://arstechnica.com/gadgets/2025/02/linux-leaders-pave-a-path-for-rust-in-kernel-while-supporting-c-veterans/.

[20] Andrew S. Tanenbaum and Herbert Bos. *Modern Operating Systems*. Pearson, 4 edition, 2015.

[21] The Rust Project Developers. The rustonomicon: Ffi – foreign function interface. https://doc.rust-lang.org/nomicon/ffi.html, 2021. Accessed: 11 de junio de 2025.

[22] Philipp Oppermann. Writing an os in rust, 2022. Accessed: 8 de junio de 2025. URL: https://os.phil-opp.com/.

[23] Rust Embedded Working Group. no_std - rust embedded book, 2023. Accessed: 2025-06-09. URL: https://docs.rust-embedded.org/book/intro/no-std.html.